Resolve user manual

Payment aftercare for merchants and buyers

Edition: 11 November 2026. All screenshots show the actual compiled application connected to FastAPI and disposable storage. Files 40-44 show an actual owner-authorized USD 1.00 official PayPal Sandbox order, capture and independently confirmed full refund. Other captures contain manually entered QA records, with providers disabled and unverified money shown unavailable. Product demo modes and fake success paths have been removed. **A public production URL has not been assigned or verified.** Follow the deployment guide before sharing as a hosted service.

1. Introduction

Resolve brings support promises, order details, payment evidence and buyer communication into one case. Use it when a customer needs a refund, return, replacement or another remedy after checkout. Merchants and support teams manage the work; buyers use private links to follow their own cases.

Implemented features include a case inbox, versioned promises, evidence checks, a shared timeline, private customer messaging, buyer offers, guided returns, saved PDF return labels, order synchronization, staff roles and reviewed dispute evidence packages. Optional integrations include PayPal Sandbox, Shopify, EasyPost, Zendesk, Google, email and advisory AI.

A customer request is not an agreed promise. A merchant-entered note is not independent provider proof. A refund approval does not submit a refund. PayPal-confirmed completion does not establish when a bank makes funds available. Resolve's payment execution is currently Sandbox only.

2. System requirements

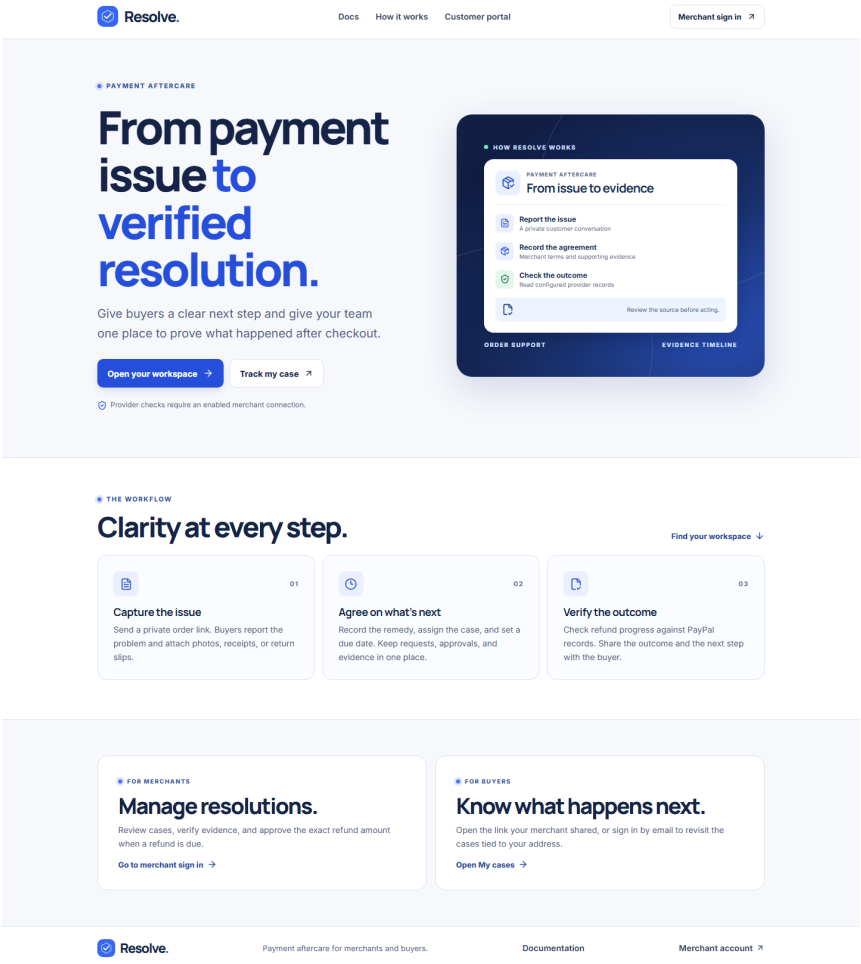
Use a current desktop or mobile browser with JavaScript, cookies and an internet connection to the hosted service. Local use requires the frontend and backend to be running. Desktop, tablet and phone layouts are included; this audit exercises widths 1440, 1180, 768 and 390 pixels. See the testing report for actual Chromium/Firefox execution results. Safari, real devices and assistive-technology certification remain unverified.

Merchants need a workspace account, a private staff invitation, or a provisioned merchant access key. Buyers need a valid private case/intake link or an invited mailbox when My cases sign-in is enabled. Optional provider accounts must be configured by the workspace owner. PDF downloads need a PDF viewer. Printing requires the browser's print dialog.

3. Getting started

Open Resolve

For the existing local preview, open `http://127.0.0.1:3000`. For a hosted release, use the exact HTTPS address provided by the project owner after deployment. This manual intentionally does not invent that address. The home page links to Docs, the customer portal and merchant sign-in. Workspace actions require the backend. Missing services produce a visible error or unavailable state.

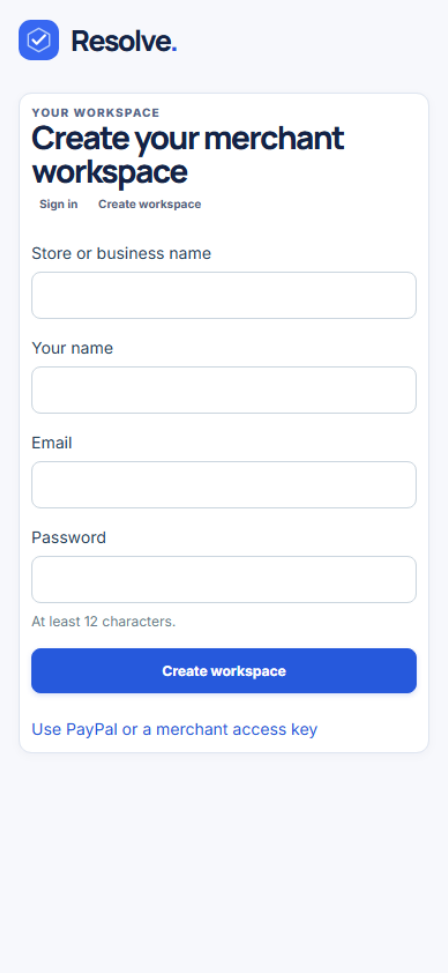


Home page with application entry points

Register a merchant workspace

1. Open `/login`, then choose “Use your staff account or create a workspace.”
2. On `/account`, select **Create workspace**.
3. Enter your store/business name, your name, email and a password of 12-256 characters.
4. Select **Create workspace**. Success opens the Operations overview; you become the owner of the new workspace.

An existing account email cannot register again. Public signup can be disabled by the administrator. To retain a legacy key-based workspace, sign in with its full key first and use `/account?claim=1` to create its first owner account instead of creating a separate business.

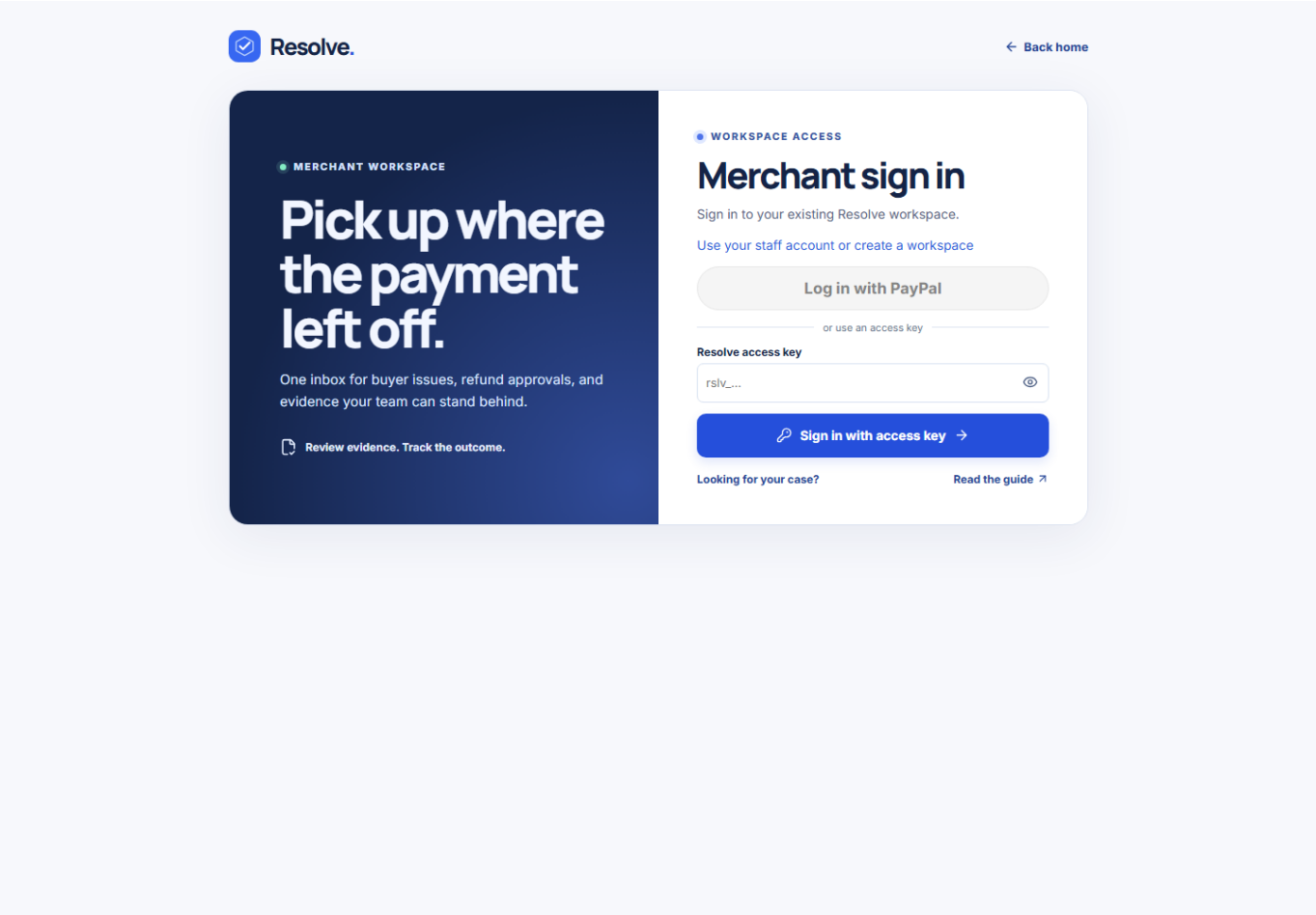


Merchant workspace registration form

Sign in and sign out

Staff use `/account` and choose **Sign in** with their email/password. A team invitation opens that page in **Join your team** mode; use the invited email and complete the form before the private invitation expires. Invitations are single use and are shared privately by the owner; automatic invite email is not implemented.

Merchants with a full access key can use `/login`: enter the key and choose **Sign in with access key**. The optional **Log in with PayPal** button works only after the owner has enabled identity login and linked the account. It does not connect payment credentials or authorize refunds.



Merchant sign-in with access key and optional PayPal

Use **Sign out** in the workspace shell/settings to end a merchant session. Sessions expire after eight hours. Role changes or account deactivation revoke staff sessions. Sign in again if redirected to `/login`. There is no password-reset page: contact the workspace owner/project administrator for access recovery; do not send your password or full key in a message.

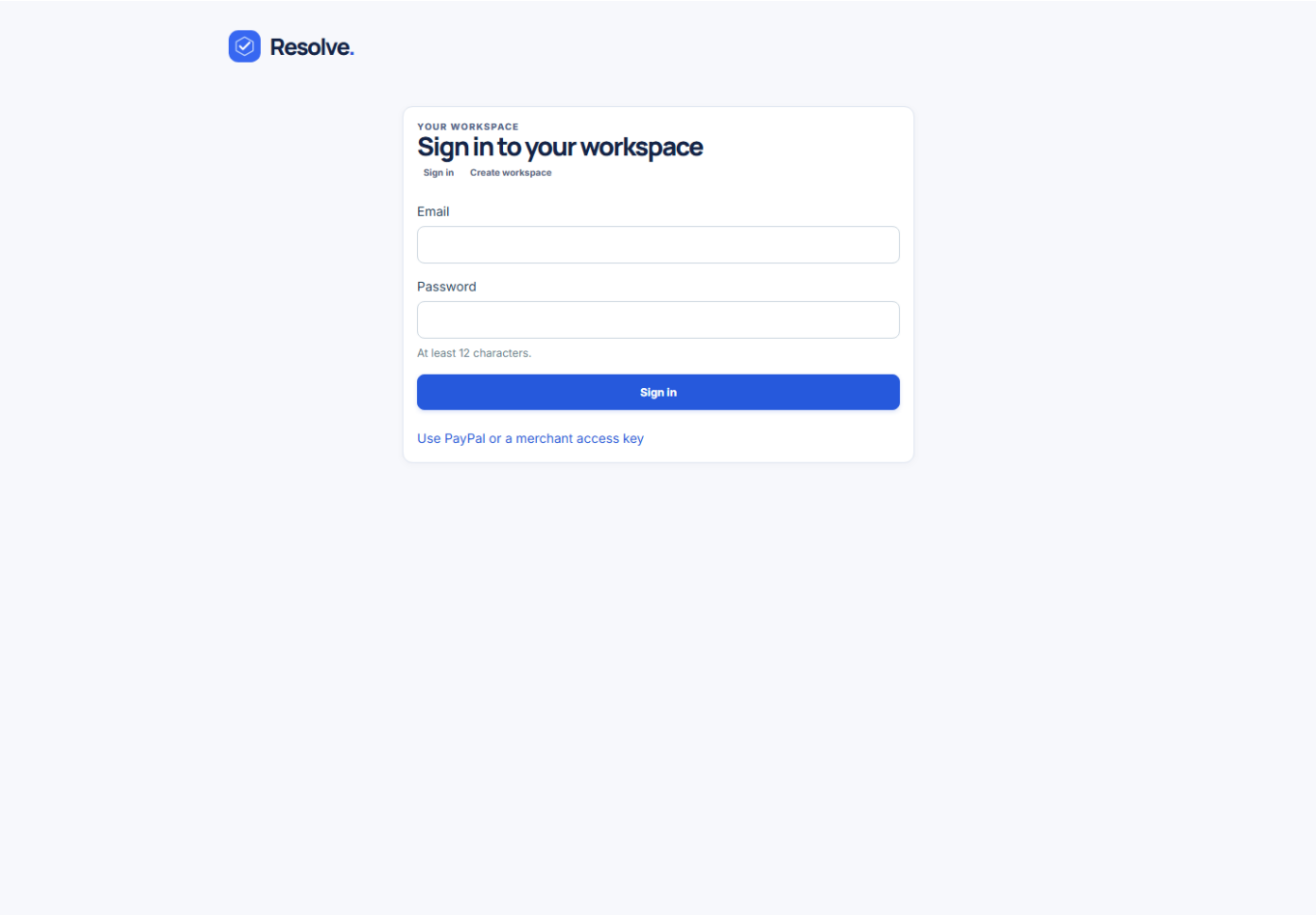
4. Complete page guide

Home - `/`

Purpose: understand the payment aftercare workflow and choose merchant, buyer or documentation entry. **Open your workspace** opens the account page; **Track my case** opens the buyer entry. The workflow card explains steps; it is not a payment receipt. **How it works** scrolls to the workflow section; **Docs** opens the searchable guide hub. Expected result: navigation to the chosen module without needing an account for public content.

Merchant account - `/account`

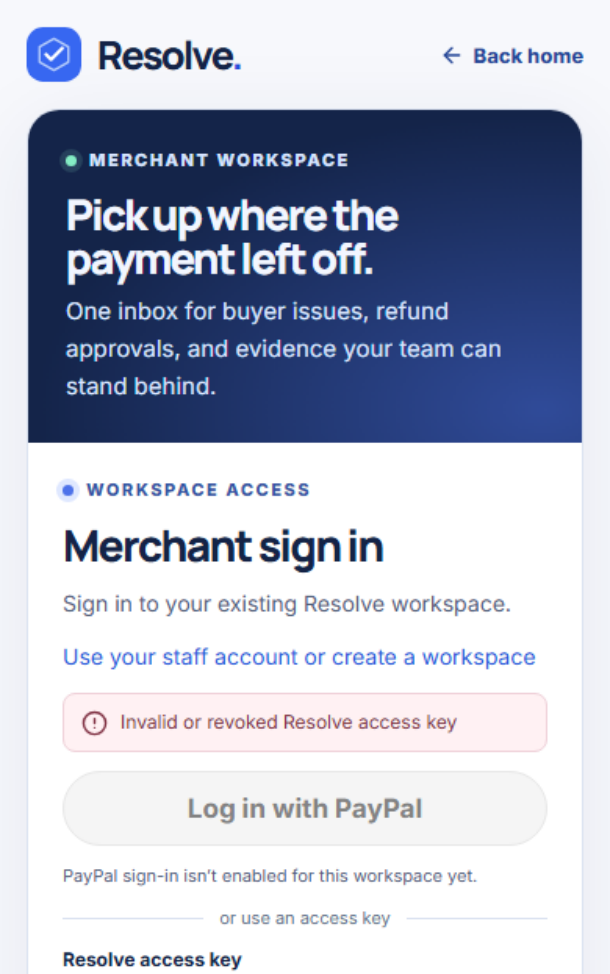
Purpose: email/password sign-in, new workspace registration, first owner claim and staff invitation acceptance. The tabs switch between sign-in and registration. Required fields and password length are checked before submitting; the server also validates them. Controls are disabled until JavaScript loads so credentials cannot be sent in a URL. Duplicate email, expired invitation, invalid login and rate-limit failures appear as a visible message. Wait for submission to finish rather than refreshing it.



Staff account sign-in

Merchant access-key login - `/login`

Purpose: authenticate an existing merchant key or linked PayPal identity. Use the eye icon to show/hide the key only in a private setting. The form displays invalid/revoked key errors and offers the staff-account page, home, buyer portal and documentation links. A disabled PayPal button means the feature is not configured. Signing in from `/login?next=settings` opens Settings directly.



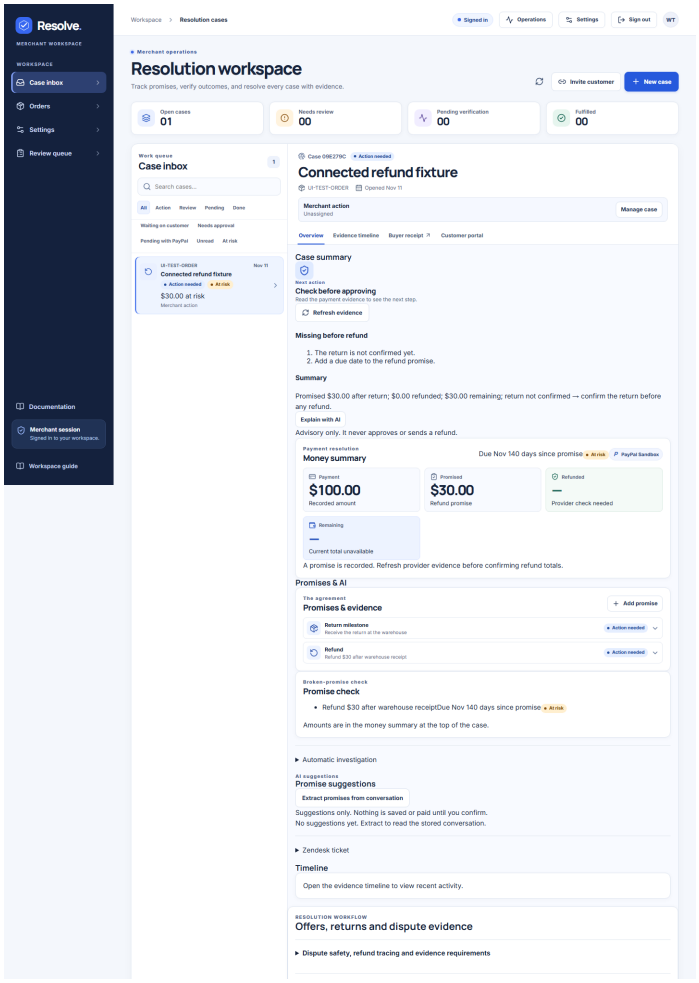
Example invalid-key error; no real credentials shown (upper portion; complete capture supplied separately)

Retired URL - `/demo`

This legacy URL redirects to `/login`. It contains no sample records or simulated actions. Old `/customer?demo=1` and `?intake=1` URLs open the normal private-link entry; a valid issued link is required.

Case inbox and case workspace - `/workspace`

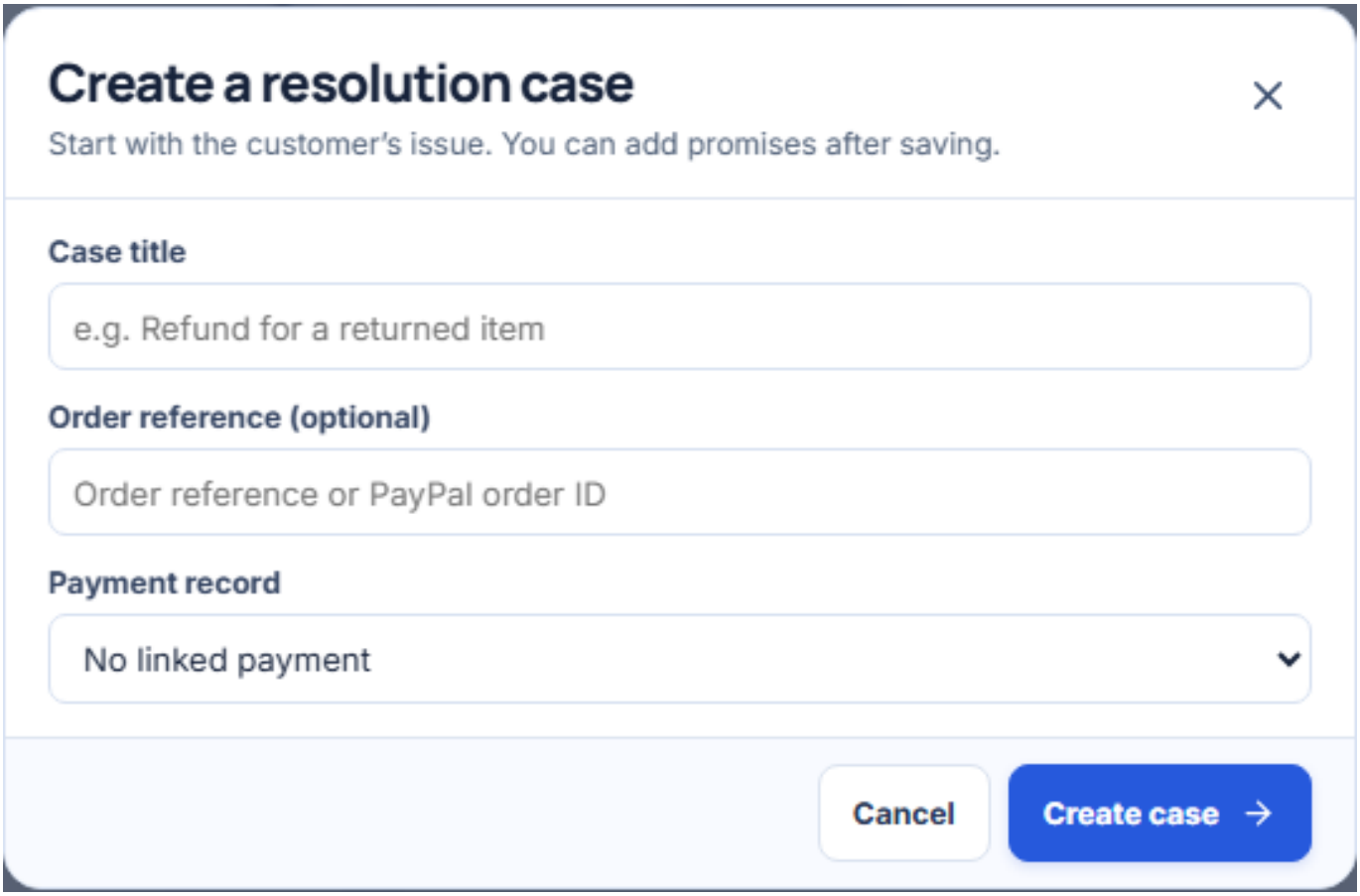
Purpose: manage the merchant's resolution cases. The sidebar links to Cases, Orders, Operations, Settings and Docs. The inbox offers text search, case/workflow/risk/review filters, refresh, new-case creation, customer invitations and the review queue. Choose a case to open its detail. `/workspace?case=UUID` opens that case directly, subject to authorization. On a phone, use the back control to return from case detail to the list.



Authenticated case inbox (upper portion; complete capture supplied separately)

The case heading identifies the issue/order and lifecycle. The overview contains current promises, status/evidence checks, financial progress when linked, and the next suggested step. Payment and evidence summaries show their source; missing history must not be interpreted as a verified zero. Secondary AI/helpdesk/operations panels open on demand.

New case: choose the new-case action, enter a title and select the payment mode. For a new payment record supply PayPal order ID, capture ID and captured USD amount; an agreed refund also needs its source. Alternatively link an owned existing payment case or create a case without a payment. Successful creation adds it to the inbox. If payment creation succeeds but the resolution case fails, retry using the retained payment reference rather than inventing a second financial record.



New-case form backed by the actual API

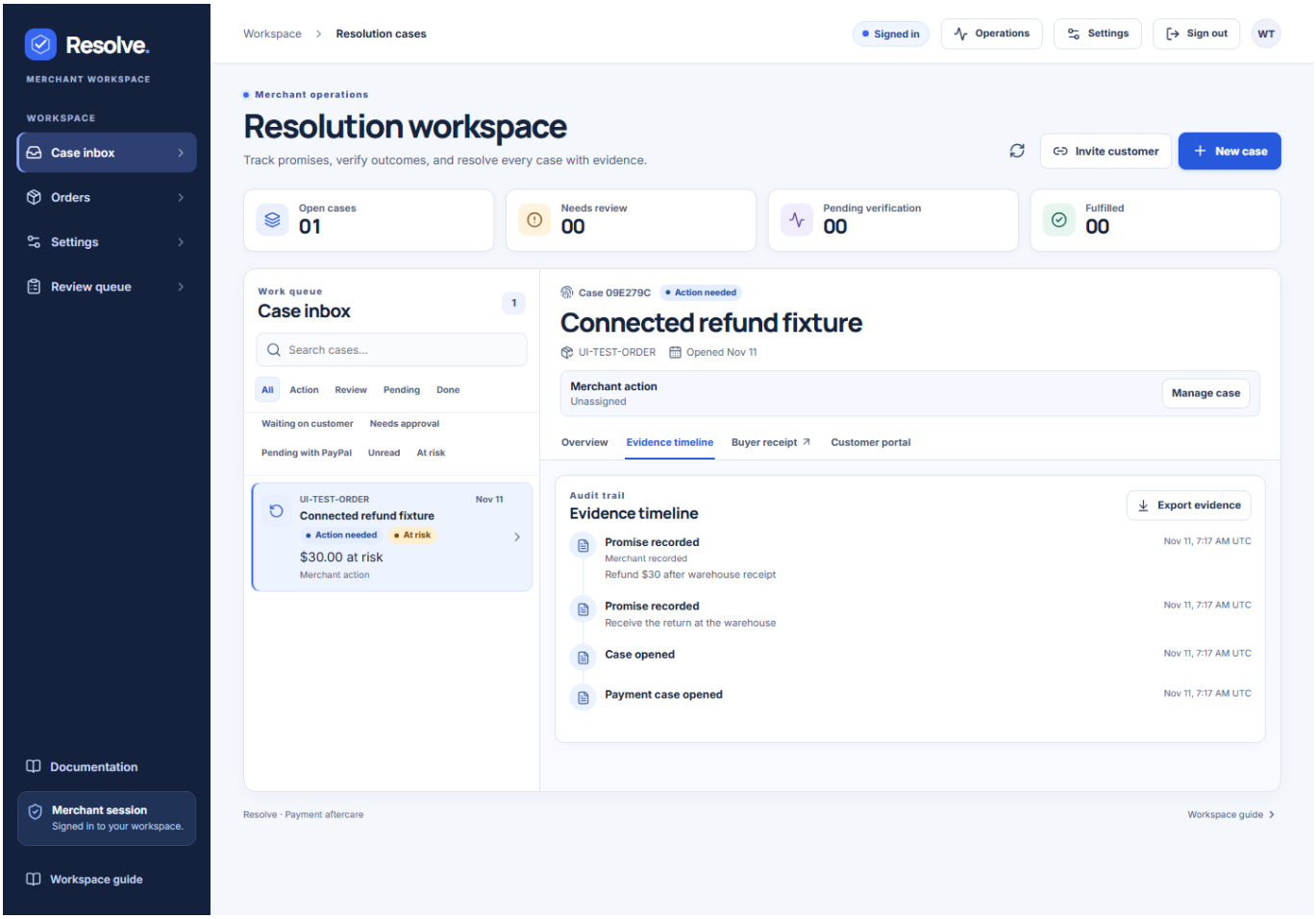
Promises: choose the add-promise action, select the remedy, enter the promise and source evidence, complete remedy-specific fields, optionally set a due date and prerequisites, then save. Editing creates a new version; it can invalidate previous evidence/approval. Only mark milestones when the evidence actually supports them.

Evidence: record a note/reference against the correct promise, or request an enabled provider refresh. Internal warehouse receipt and inspection remain merchant-recorded. A provider outage or missing record should leave an evidence gap for review, not be marked fulfilled.

Case management: assignment labels, deadlines, workflow status and close/reopen notes help triage. Closure is a case lifecycle action and does not prove a payment completed. Existing audit/history is retained; a general delete-case feature is not provided.

Case detail: Evidence timeline

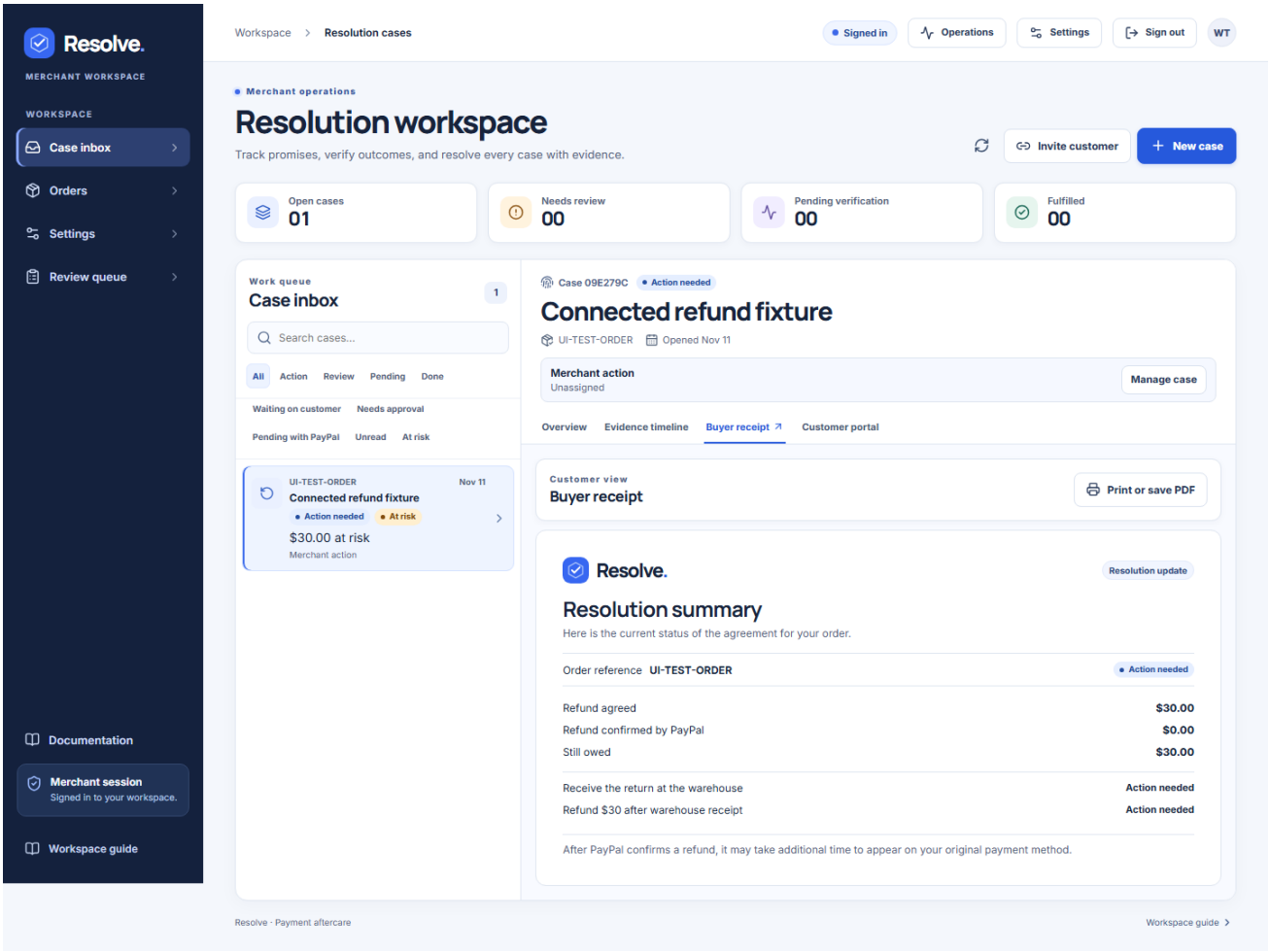
Choose **Evidence timeline**. Review timestamped promise, evidence and financial events with their sources. Load older events where available. Returning to the tab reuses loaded timeline data; explicitly refresh when checking new evidence. Exported data can contain private source material; share only with authorized recipients.



Evidence timeline of persisted QA case events

Case detail: Buyer receipt

Choose **Buyer receipt** to inspect the printable summary and print/download using the available browser actions. Check what is complete, pending, outstanding or blocked. Verify the source/time of payment evidence before presenting a receipt as a provider-confirmed outcome. Printing opens the browser dialog; choose a printer or Save as PDF.



Buyer receipt with unverified provider totals clearly unavailable

Case detail: Customer portal

Choose **Customer portal**, enter a customer heading, optionally expose the order reference, and choose **Create customer link** or **Replace customer link**. Copy the private link and share through your approved support channel. Treat the link as a credential. Review buyer messages, write a reply and select **Send message**. Use **Refresh messages** to check new replies. **Revoke access** removes access; replacement/revocation and account sign-out can invalidate existing links.

Case detail: Offers, returns and dispute evidence

Open **Offers, returns and dispute evidence** in the case overview. Expand the section you need.

• **Buyer resolution offers:** select current promise versions, set an expiry and publish an option. Buyers accept or decline from their private portal. Revised, replaced or expired offers require a new version; a refund offer's current acceptance is an additional financial gate.

• **Guided returns and replacements:** choose synced order items/quantities and record review, tracking, receipt, inspection and completion steps with evidence in order. Requests reserve quantities to prevent over-returns. Physical shipping remains arranged outside Resolve.

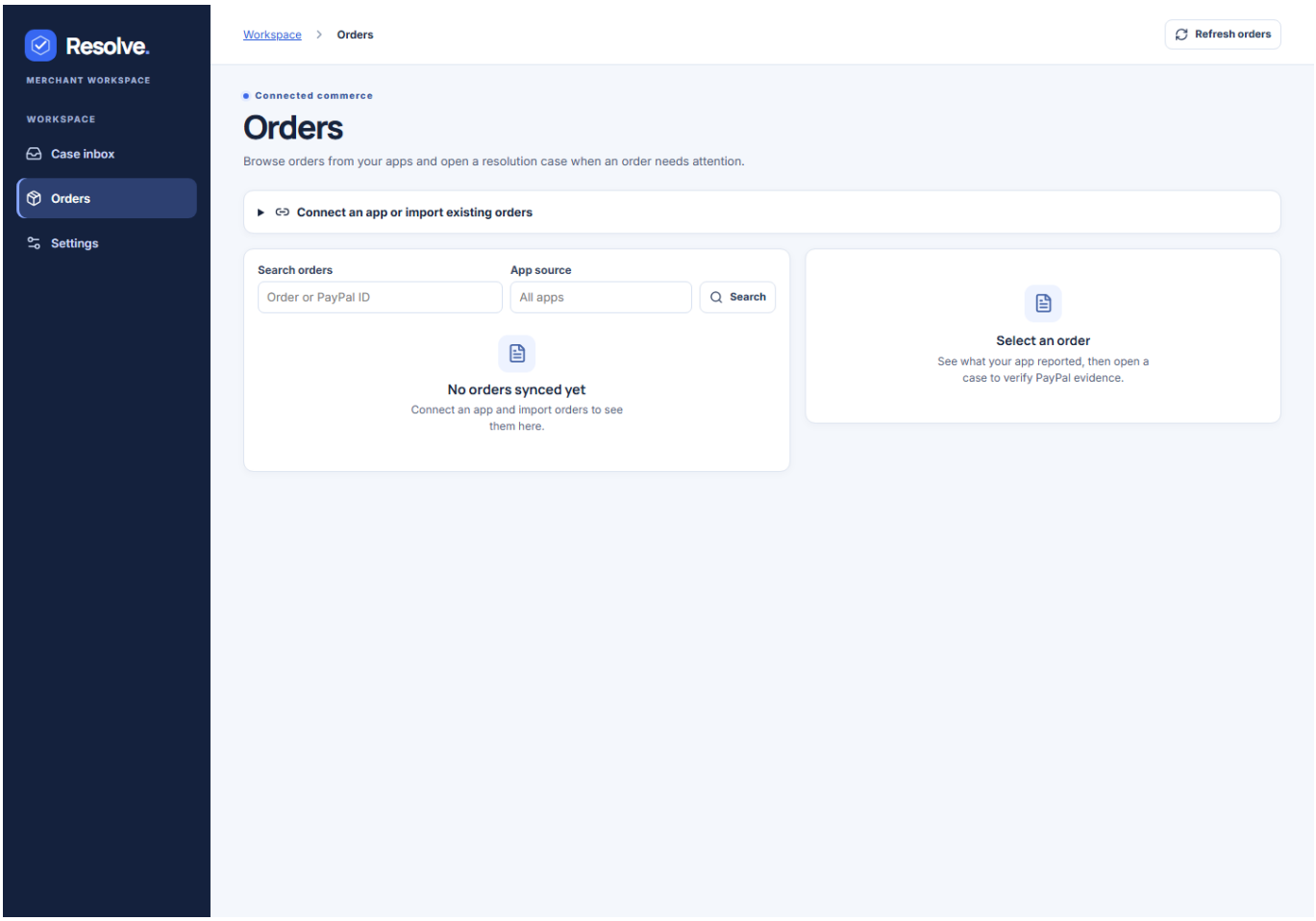
• **Return instructions, saved labels and inspection:** save return text/address and an existing PDF label; record inspection against the correct return. Uploading a label stores an existing file; it does not buy postage.

• **Dispute safety, refund tracing and evidence requirements:** authorized staff record the dispute state/reference and evidence. An active reimbursement/dispute hold blocks further capture refunds. Finance/owners can add a recorded refund trace such as an ARN. Private tracing notes are not buyer messages.

• **Dispute evidence packages:** enter reference/reason, prepare the snapshot, inspect its missing-evidence checklist, have an owner/manager record review, then export JSON. A package is not a PayPal dispute submission. New evidence needs a newly prepared and reviewed package.

Orders - `/workspace/orders`

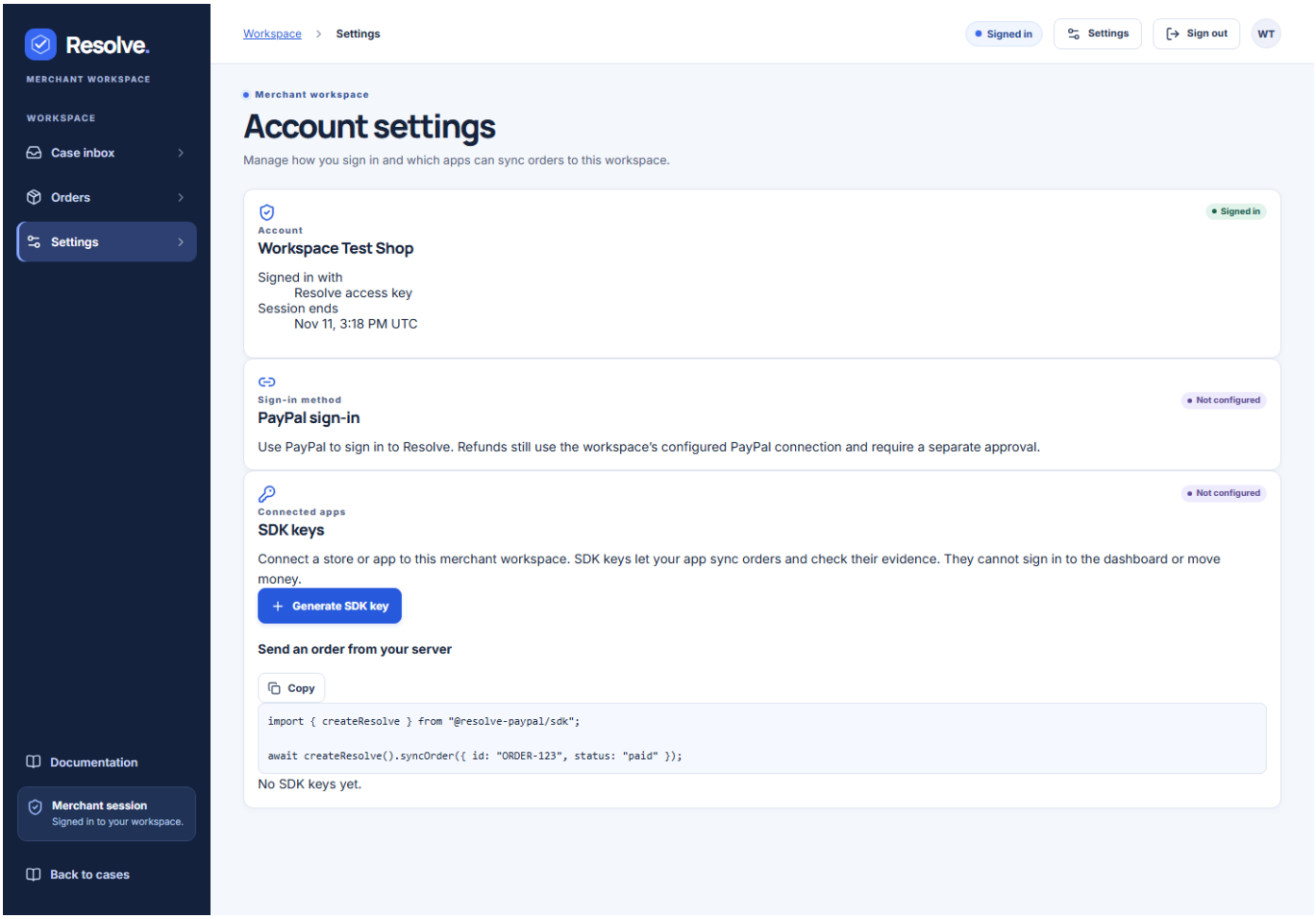
Purpose: browse app-registered orders, including those without cases. Search by store or PayPal reference and select a result. The detail shows registered item/payment data and any comparison. **Create resolution case** opens support for a selected order; **Open case** navigates to an existing linked case. Registration is performed by the merchant's server/SDK, not a public checkout on this page. No result means no matching registered order, not proof that the purchase never occurred.



Orders list and search

Settings - `/workspace/settings`

Purpose: inspect signed-in workspace/session, optional PayPal identity link and scoped SDK keys. Only authorized owners can issue/revoke keys. **Generate SDK key** displays server setup once; copy it into your app server's secret store and then **Hide key**. Never place it in browser code or a recording. Confirm revocation to disable a compromised integration key. The code-copy control copies the SDK example; it does not execute it. PayPal linking/unlinking requires the supported key-authenticated flow and is separate from payment connections.

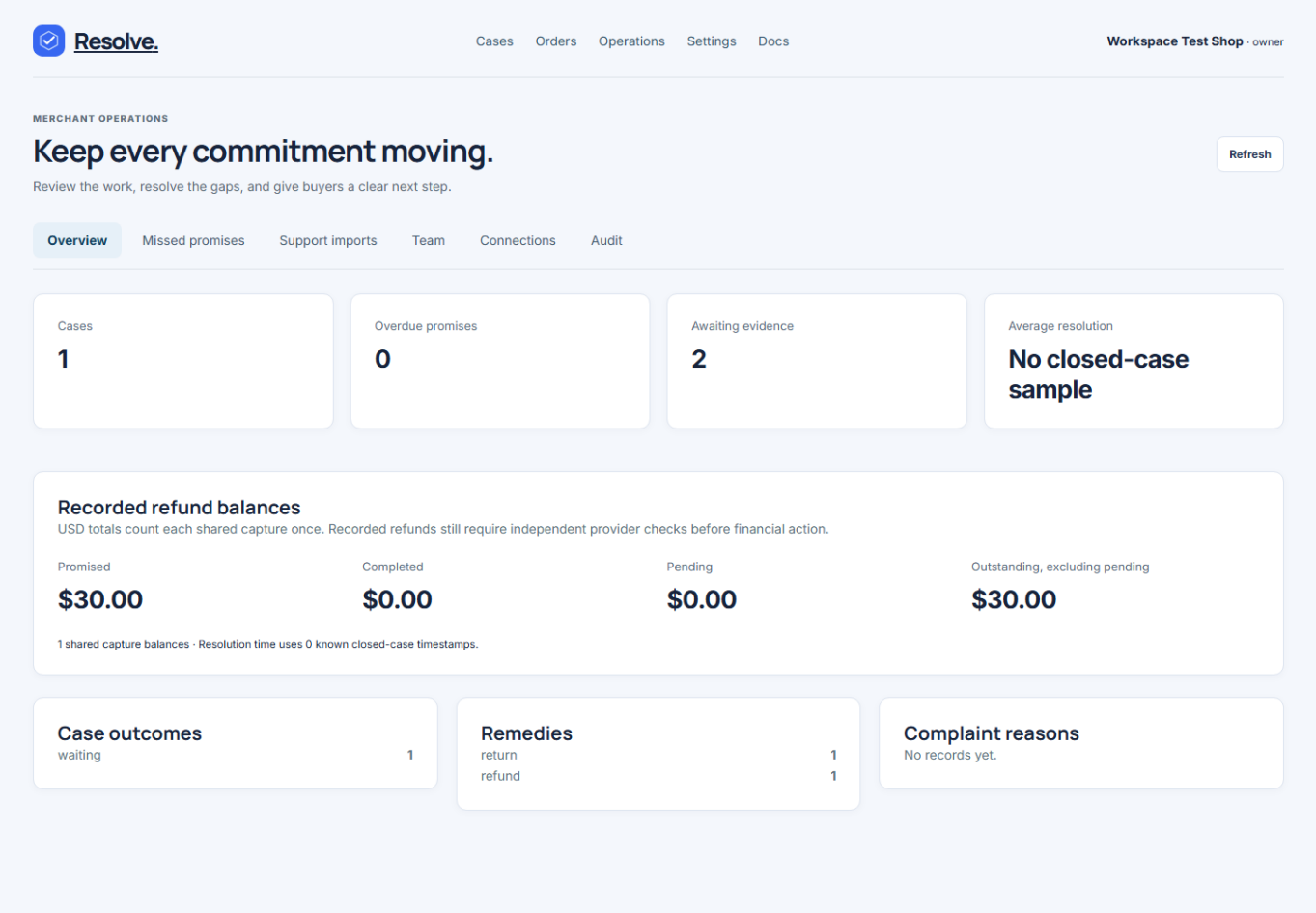


Settings with no exposed integration key

Operations - `/workspace/operations`

Purpose: coordinate the whole merchant workspace. Choose its six section tabs; they share one URL.

Overview: case outcomes, overdue work, complaint categories, gaps, closure sample times and recorded USD refund balances. Shared captures count once. Recorded totals and provider proof have different meanings; closure times include their sample count.



Operations overview

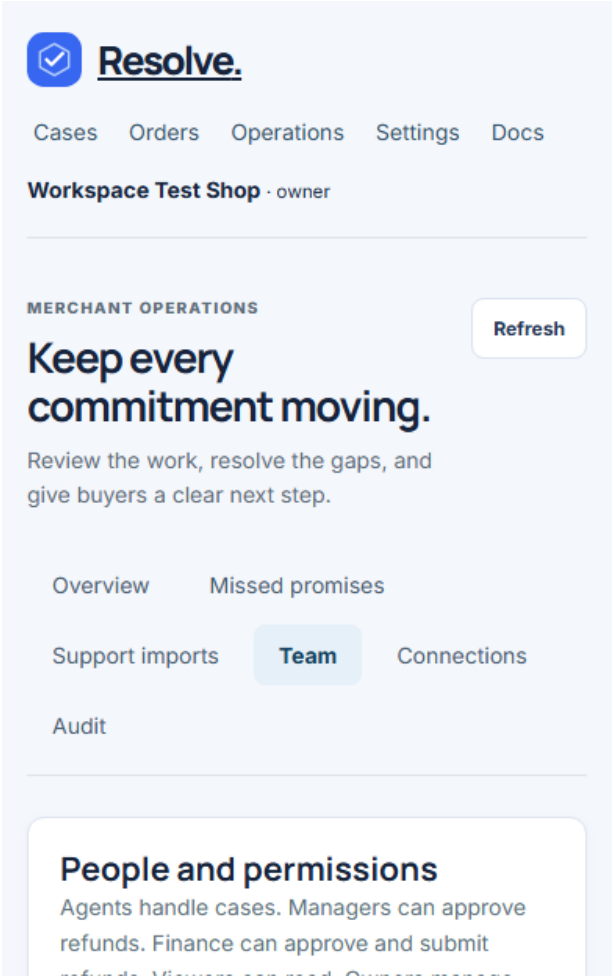
Missed promises: select **Scan workspace** for a current pass, review alerts, and copy an update draft. The scan does not send the draft or issue refunds. The operations worker maintains periodic scans when deployed.

Support imports: enter a Zendesk ticket ID to import with an enabled connection, review the exact reference match and select/confirm the correct case if ambiguous. Private source comments stay private. Failed imports need a connection/permission check; do not link to a guessed order.

Team: owners enter staff email/role and create a private invitation. Share it privately. Review role/active controls; changing access ends existing staff sessions. At least one active owner must remain.

Connections: owners configure enabled provider connections, retain saved secrets by leaving those fields blank, and run **Check account access**. A credential check does not prove capture ownership. Return policy controls set review/fee terms. Support integration keys bind to an app source and can expose the enabled aftercare workflow to that app's authenticated server.

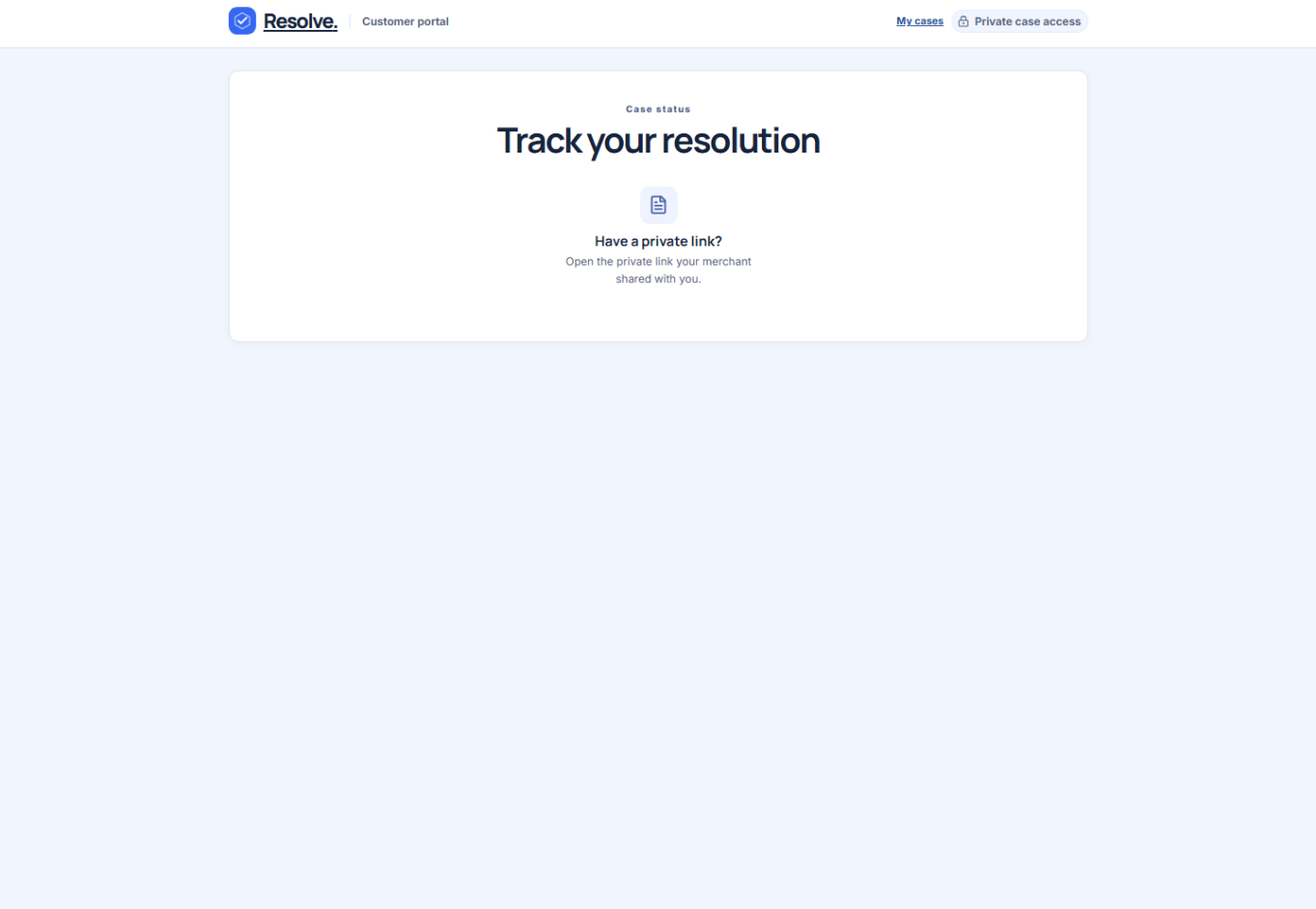
Audit: inspect actor-attributed workspace actions and timestamps. The engineering log is a development record, separate from these runtime events.



Operations Team section (upper portion; complete capture supplied separately)

Buyer entry and private portal - `/customer`

Purpose: open a private order/case link. Paste the complete link or token in the entry control when needed. A valid intake link opens the issue form; choose the issue type, describe the problem and submit once. This reports a request, not an accepted remedy. Private case links show safe progress, messages and configured actions. Invalid, expired or revoked links display an error; request a new link from the merchant.

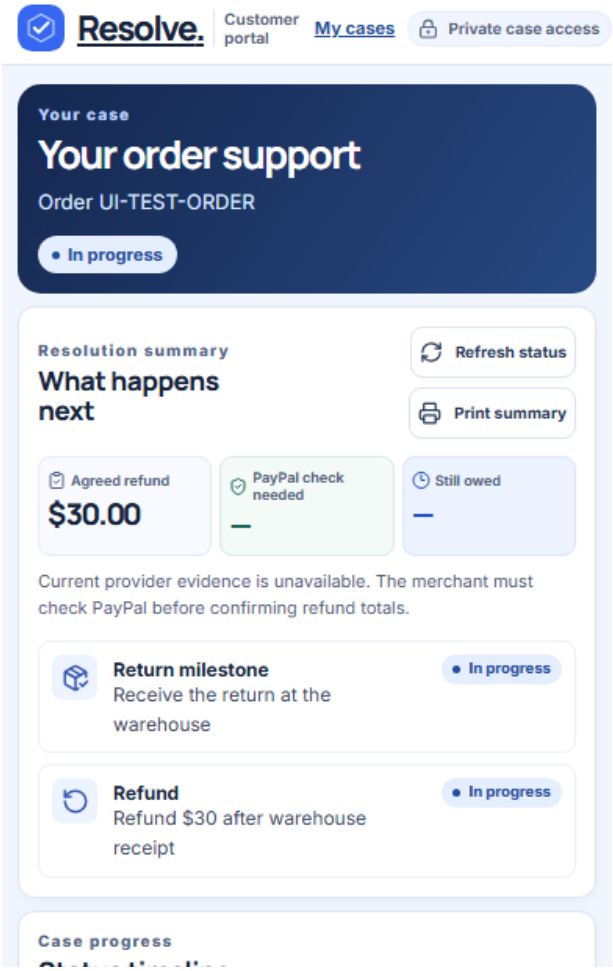


Buyer entry page

Status and messages: read the remedy, current milestones and next action; write a message and select **Send message**. Use **Refresh status** or **Refresh messages** to retrieve changes. Add supported evidence files through the attachment control; observe type/size errors rather than renaming unsupported content.

Offers: review the exact terms/conditions and expiry before choosing **Accept this option** or declining. Acceptance is recorded for that version and does not itself submit a payment.

Returns: where synced items/policy permit it, select quantities, enter a reason and **Request return**. Read the saved instructions/address and **Save label PDF** when available. Keep the carrier receipt. Merchant inspection and reimbursement updates appear when recorded; do not assume a requested return is already approved.



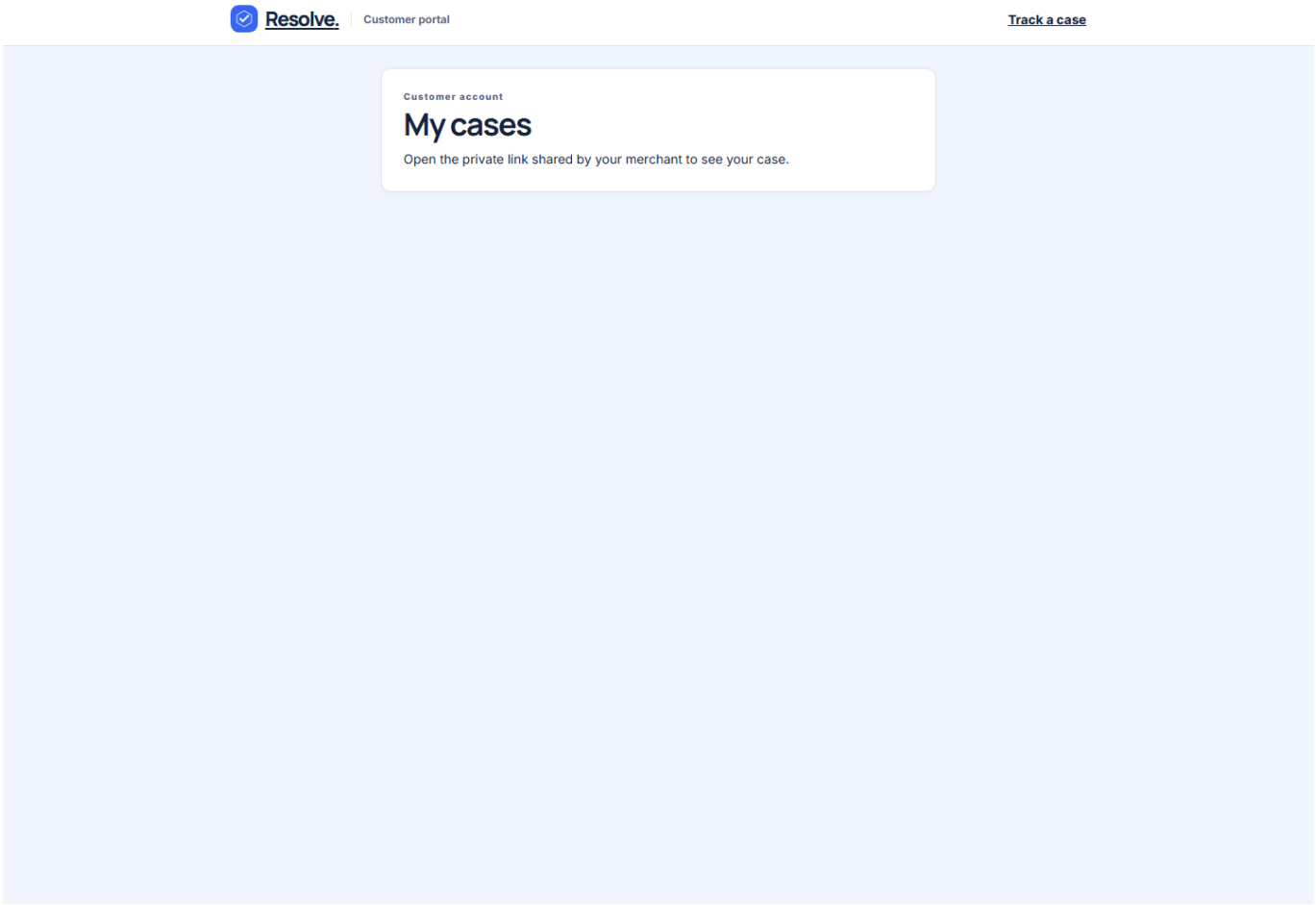
Private buyer case with actual persisted messages and honest unavailable provider state (upper portion; complete capture supplied separately)

Refund tracking: "Where is your refund?" distinguishes reported state, trace reference and bank receipt. Use any supplied trace with your merchant/provider; Resolve cannot independently verify a bank credit.

My cases - `/customer/account`

Purpose: find cases already associated with your invited mailbox. If email sign-in is configured, enter the invited email and request a sign-in link, open it, then choose **Continue to My cases**. If Google is enabled, choose **Sign in with Google** and use the matching verified mailbox. Open a listed case with **Open case**. **Sign out** ends the customer session. These credentials cannot access the merchant workspace.

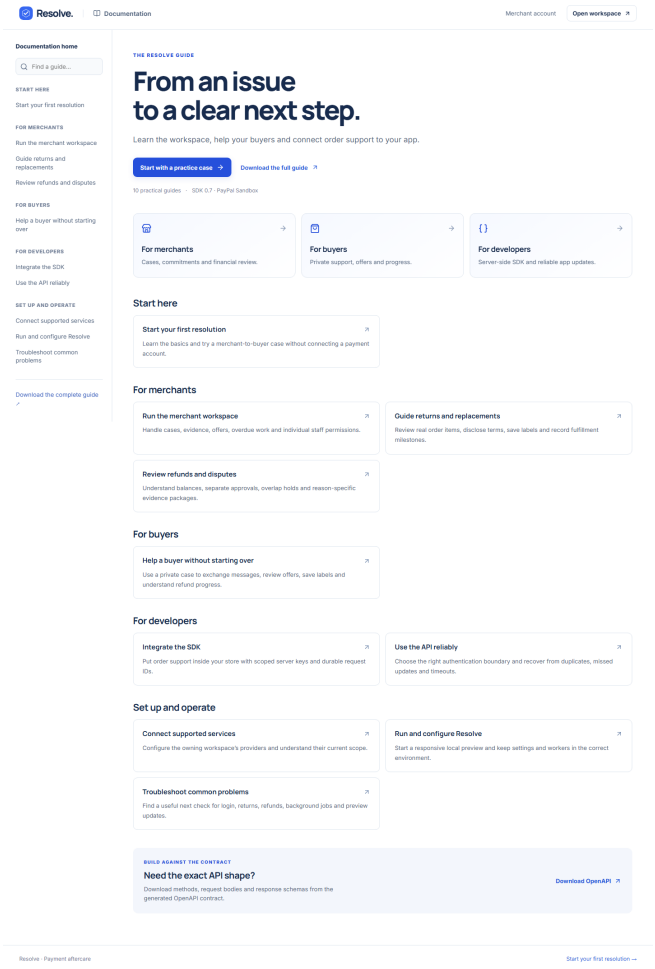
If neither option is configured, use your private case link. An empty list can mean no case was invited to that address. This page does not register arbitrary buyers or expose every order. There is no buyer password or forgot-password flow.



Buyer account entry with optional methods dependent on configuration

Documentation hub - `/docs`

Purpose: find user/developer guides without signing in. Search guide text locally; no match displays a clear state. Choose an article from the sidebar or the phone selector. The hub downloads the complete Markdown guide and OpenAPI JSON. Code blocks have **Copy code**; article print controls use your browser's print dialog. Anchors open individual sections. These downloads contain public reference material, not your cases.



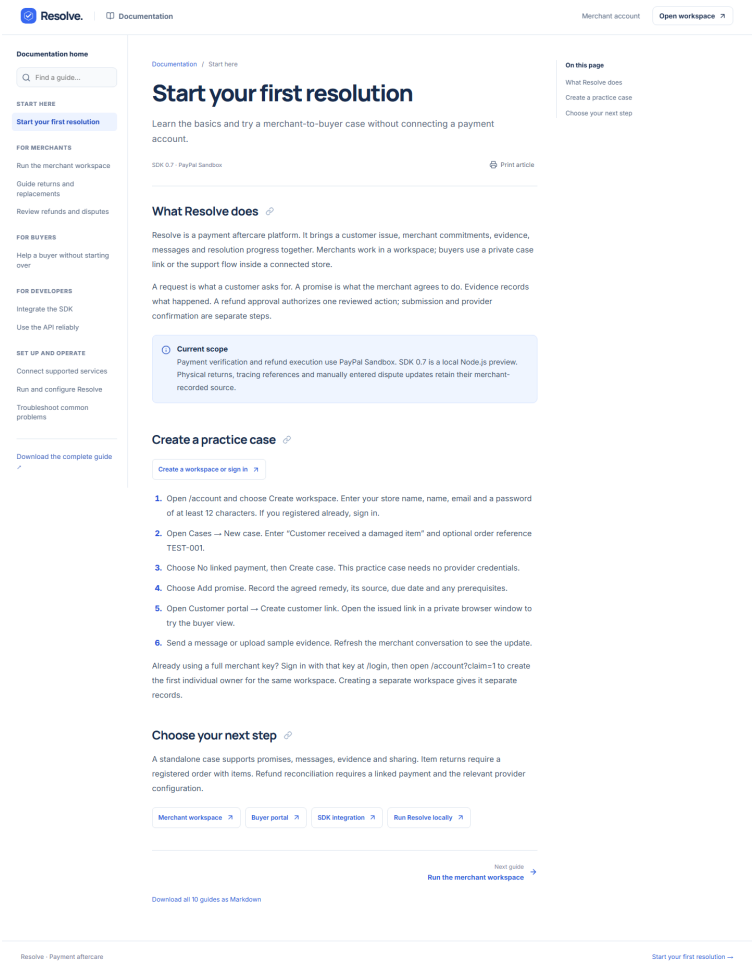
Public searchable documentation (upper portion; complete capture supplied separately)

The following ten article pages are part of the application. Each explains its topic with steps, notes, code where relevant and links to related guides. Use the article's section navigation to reach a topic; verify configuration before following optional-provider examples. An unknown guide slug returns a not-found page.

Guide: Start your first resolution - `/docs/getting-started`

Learn the basics and try a merchant-to-buyer case without connecting a payment account.

Open Docs and select this guide. Read its sections: What Resolve does; Create a practice case; Choose your next step. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

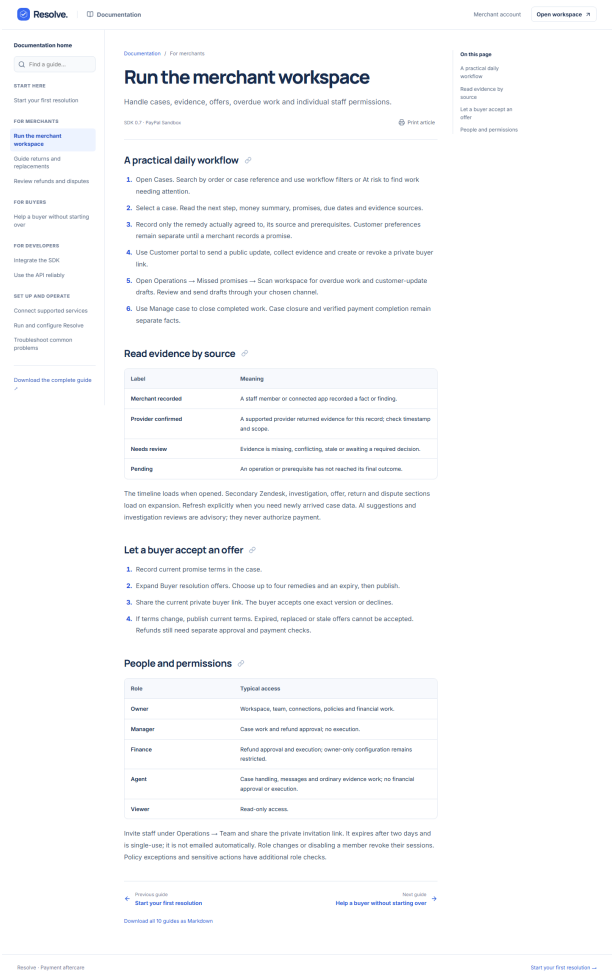


Start your first resolution - actual public article

Guide: Run the merchant workspace - `/docs/merchant-workspace`

Handle cases, evidence, offers, overdue work and individual staff permissions.

Open Docs and select this guide. Read its sections: A practical daily workflow; Read evidence by source; Let a buyer accept an offer; People and permissions. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

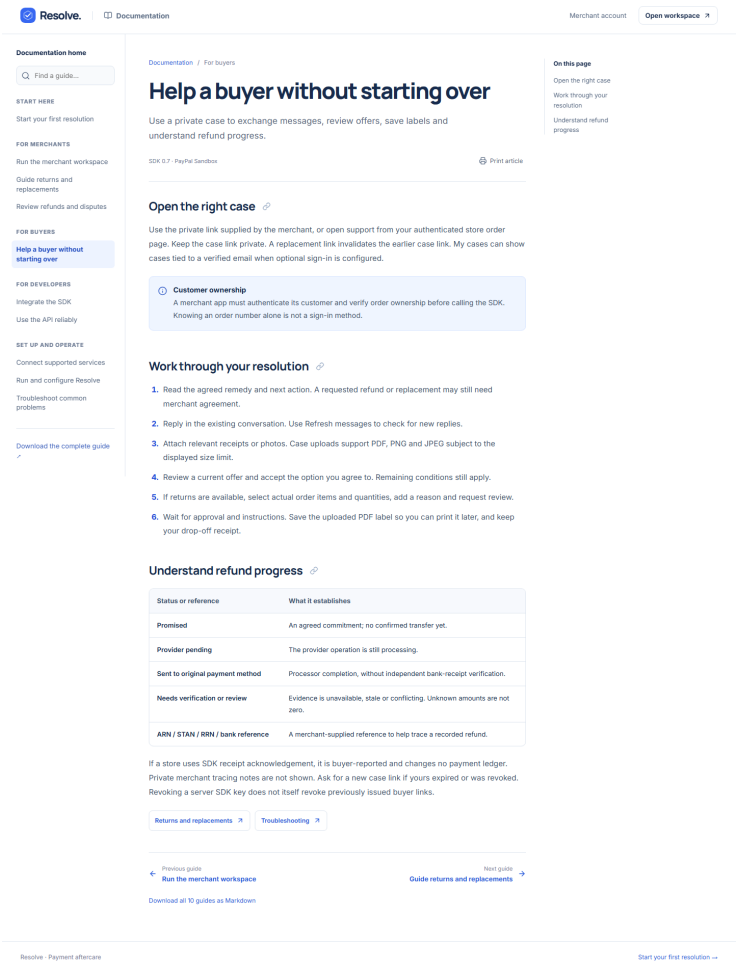


Run the merchant workspace - actual public article (upper portion; complete capture supplied separately)

Guide: Help a buyer without starting over - `/docs/buyer-portal`

Use a private case to exchange messages, review offers, save labels and understand refund progress.

Open Docs and select this guide. Read its sections: Open the right case; Work through your resolution; Understand refund progress. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

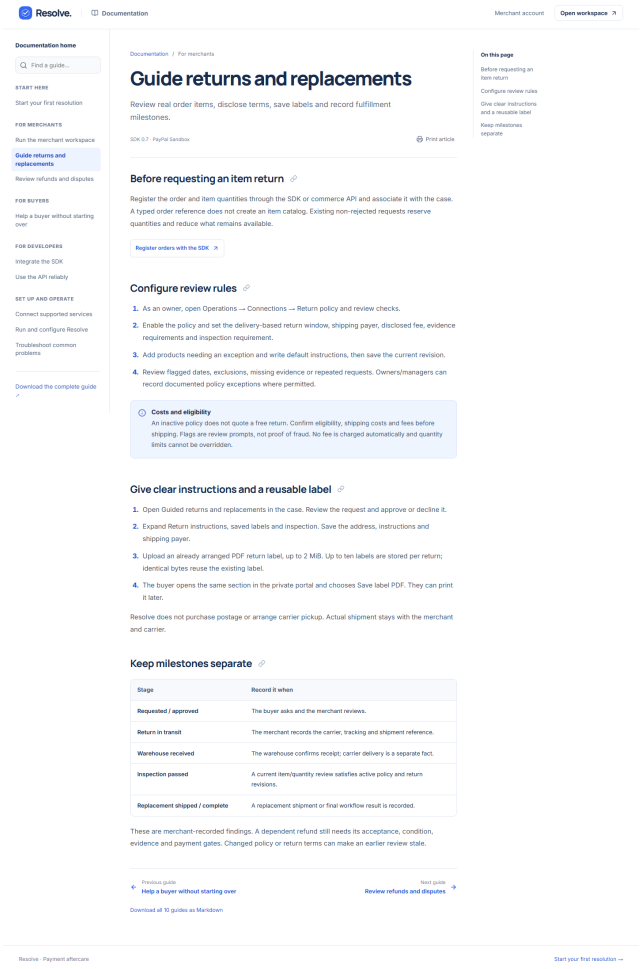


Help a buyer without starting over - actual public article

Guide: Guide returns and replacements - `/docs/returns-and-replacements`

Review real order items, disclose terms, save labels and record fulfillment milestones.

Open Docs and select this guide. Read its sections: Before requesting an item return; Configure review rules; Give clear instructions and a reusable label; Keep milestones separate. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

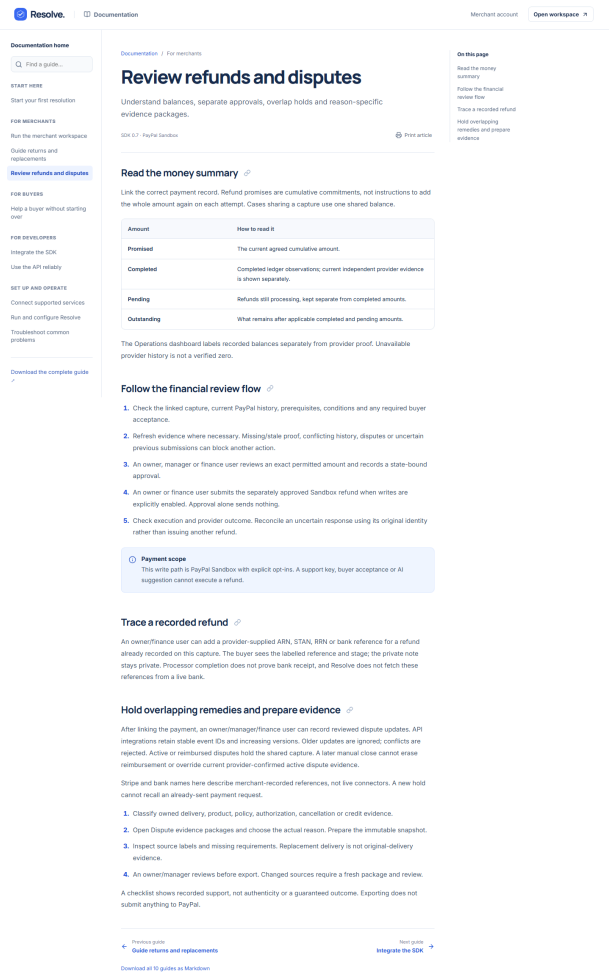


Guide returns and replacements - actual public article (upper portion; complete capture supplied separately)

Guide: Review refunds and disputes - `/docs/refunds-and-disputes`

Understand balances, separate approvals, overlap holds and reason-specific evidence packages.

Open Docs and select this guide. Read its sections: Read the money summary; Follow the financial review flow; Trace a recorded refund; Hold overlapping remedies and prepare evidence. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

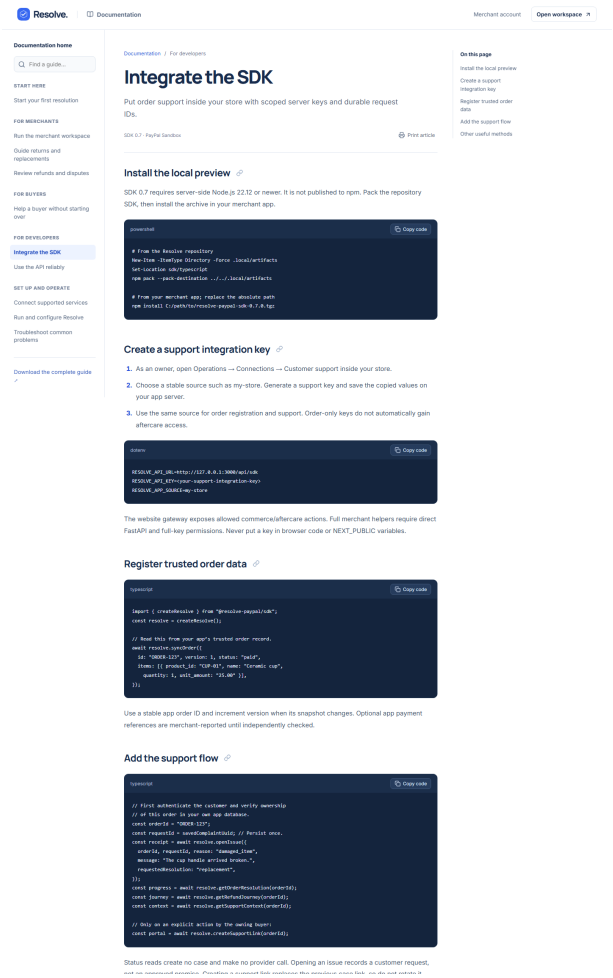


Review refunds and disputes - actual public article (upper portion; complete capture supplied separately)

Guide: Integrate the SDK - `/docs/sdk-integration`

Put order support inside your store with scoped server keys and durable request IDs.

Open Docs and select this guide. Read its sections: Install the local preview; Create a support integration key; Register trusted order data; Add the support flow; Other useful methods. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

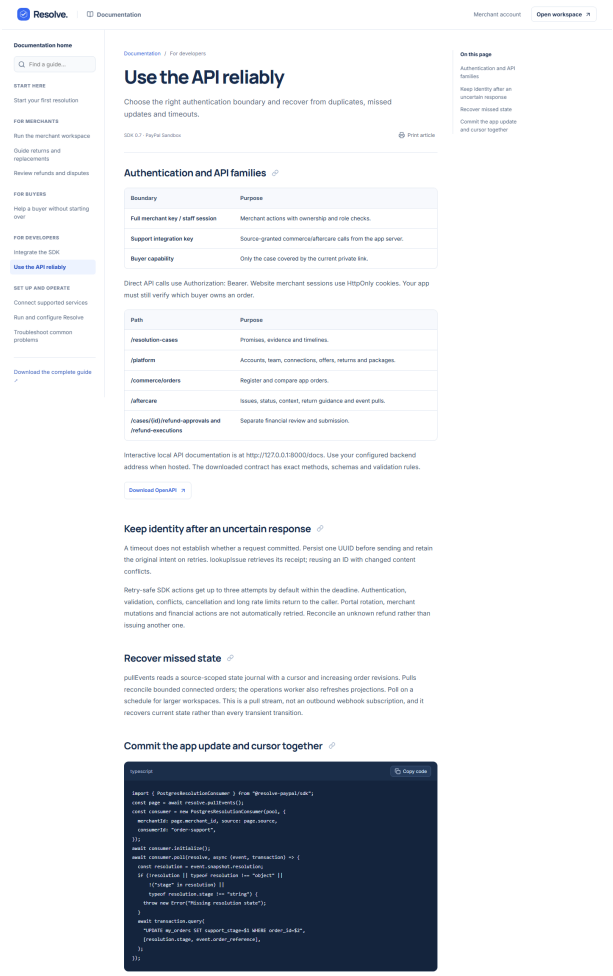


Integrate the SDK - actual public article (upper portion; complete capture supplied separately)

Guide: Use the API reliably - `/docs/api-and-reliability`

Choose the right authentication boundary and recover from duplicates, missed updates and timeouts.

Open Docs and select this guide. Read its sections: Authentication and API families; Keep identity after an uncertain response; Recover missed state; Commit the app update and cursor together. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

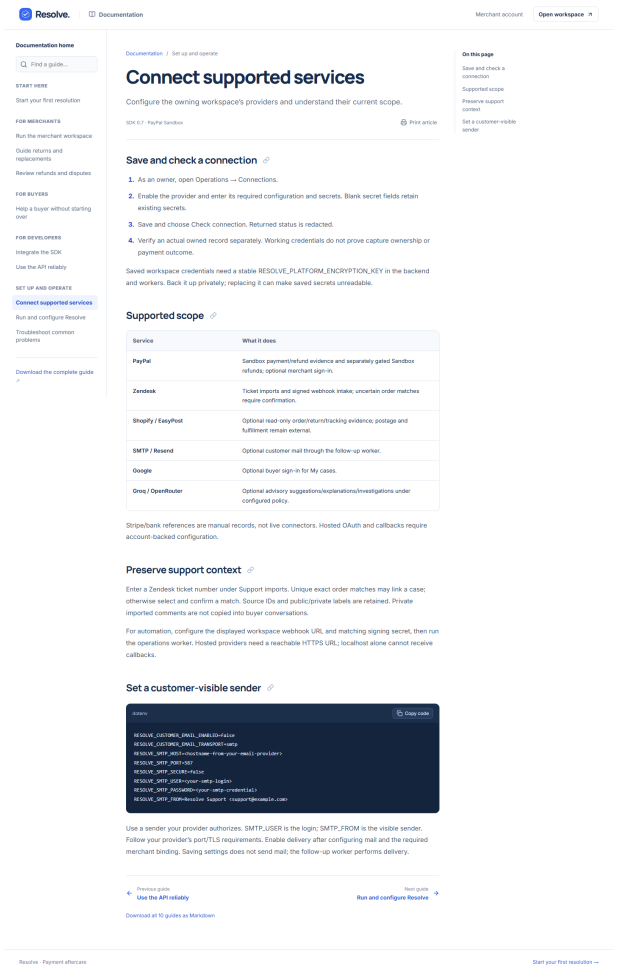


Use the API reliably - actual public article (upper portion; complete capture supplied separately)

Guide: Connect supported services - `/docs/connections`

Configure the owning workspace's providers and understand their current scope.

Open Docs and select this guide. Read its sections: Save and check a connection; Supported scope; Preserve support context; Set a customer-visible sender. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.

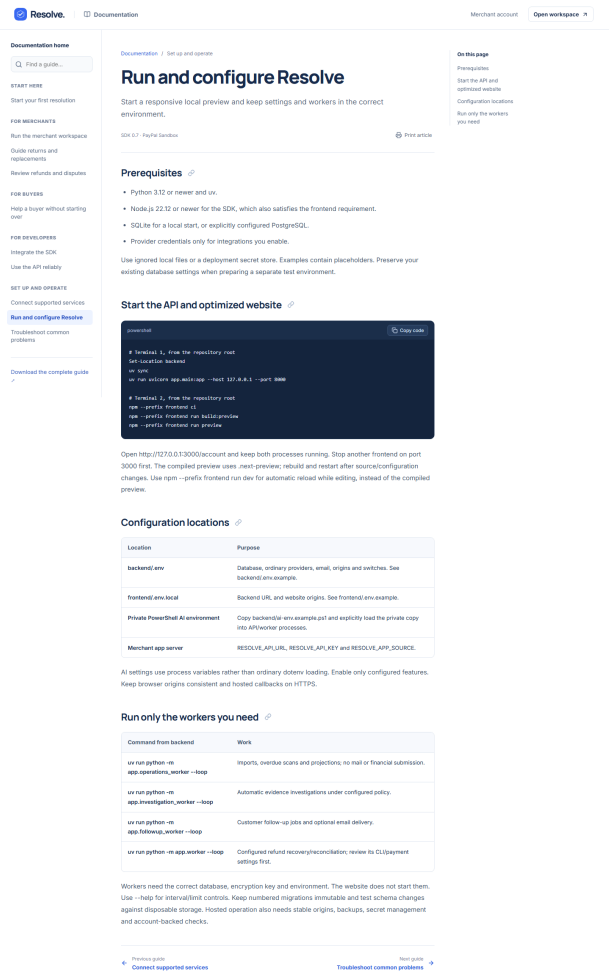


Connect supported services - actual public article (upper portion; complete capture supplied separately)

Guide: Run and configure Resolve - `/docs/running-resolve`

Start a responsive local preview and keep settings and workers in the correct environment.

Open Docs and select this guide. Read its sections: Prerequisites; Start the API and optimized website; Configuration locations; Run only the workers you need. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.



Run and configure Resolve - actual public article (upper portion; complete capture supplied separately)

Guide: Troubleshoot common problems - `/docs/troubleshooting`

Find a useful next check for login, returns, refunds, background jobs and preview updates.

Open Docs and select this guide. Read its sections: Login and support access; Orders, returns and evidence; Refund progress and timeouts; Workers and AI; Website updates. Use section anchors, Copy code where provided and Print to retain an article. Expected result: the matching public reference opens without merchant sign-in. Configuration examples do not activate an integration automatically.



Troubleshoot common problems - actual public article (upper portion; complete capture supplied separately)

Separate example store - Field Brew Supply

`examples/checkout-store` is a separate Node store, not a main application route or a deployed Render service. Its catalog, cart and private receipt use real store routes; checkout calls the official PayPal Sandbox Orders API through `src/paypal.mjs`, and its server calls the Resolve SDK. There is no product mock mode. It needs its own database and server-side PayPal/Resolve configuration. Buyer approval occurs on PayPal, then the existing adapter reads and captures the same order with stable identity. The authorized USD 1 integration audit used this adapter and the SDK, verified actual capture and full refund through the guarded backend, and confirmed persistence. It did not exercise the complete store cart in that real-service run; cart behavior has separate controlled tests. Follow its README before using it. Shipping, postage purchase and order cancellation are not implemented. Earlier mock-provider test screenshots have been excluded from this manual.

5. Practical feature tutorials

Handle an ordinary support case

1. Create a case or open a registered order's case.
2. Read the buyer's request and establish the actual agreement.
3. Add the promise with source evidence, due date and required milestones.
4. Record or retrieve the relevant evidence. Review gaps and errors.
5. Share the private customer portal, send a factual update and refresh to confirm it persisted.

6. Complete the supported remedy steps and inspect the receipt. Close the case with a meaningful note when appropriate.

Expected result: an owned case with a readable history. Reopening preserves prior history. There is no general destructive delete workflow.

Correct a promise

Open the relevant promise's edit action, correct the actual terms/source and save a new version. Recheck dependent milestones, current buyer offer/acceptance and financial approval before proceeding. A new version can invalidate earlier gates; do not treat an old approval as reusable.

Review a Sandbox refund

1. Link the correct payment and verify capture amount, currency, seller and current provider history with an enabled Sandbox connection.

2. Establish the accepted refund amount and satisfy current prerequisite/offer conditions.

3. Review the deterministic completed/pending/outstanding balance. Pending, ambiguous, stale or dispute-held outcomes can block action.

4. An owner, manager or finance user reviews an explicit refund amount and decision note to create a time-limited approval.

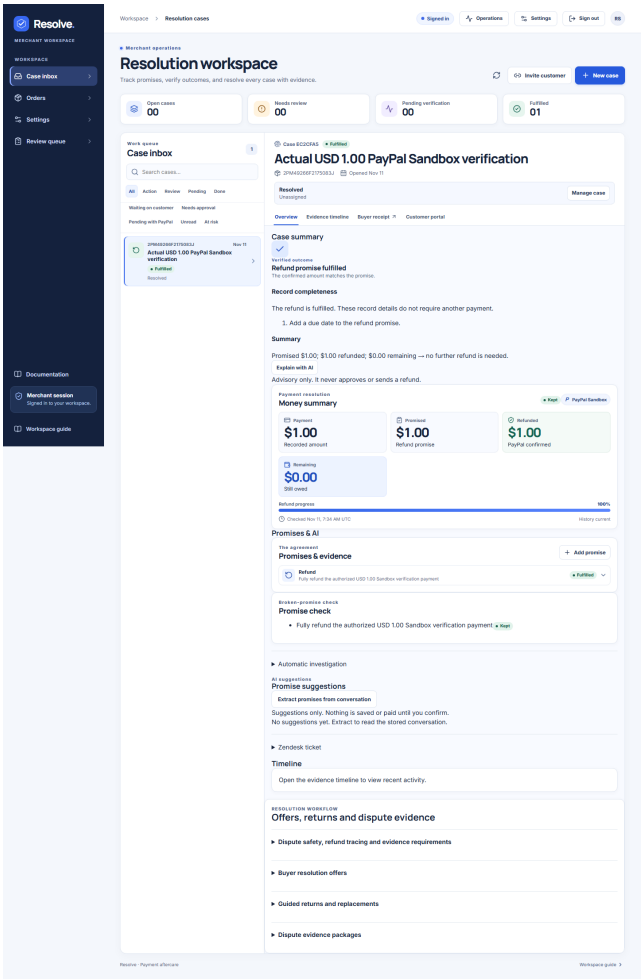
5. Only owner/finance may separately review and submit an eligible Sandbox execution. The application checks state again; a changed case invalidates approval.

6. Refresh independently read provider evidence and investigate any pending/uncertain state before retrying. Never represent this as a live-money demonstration.

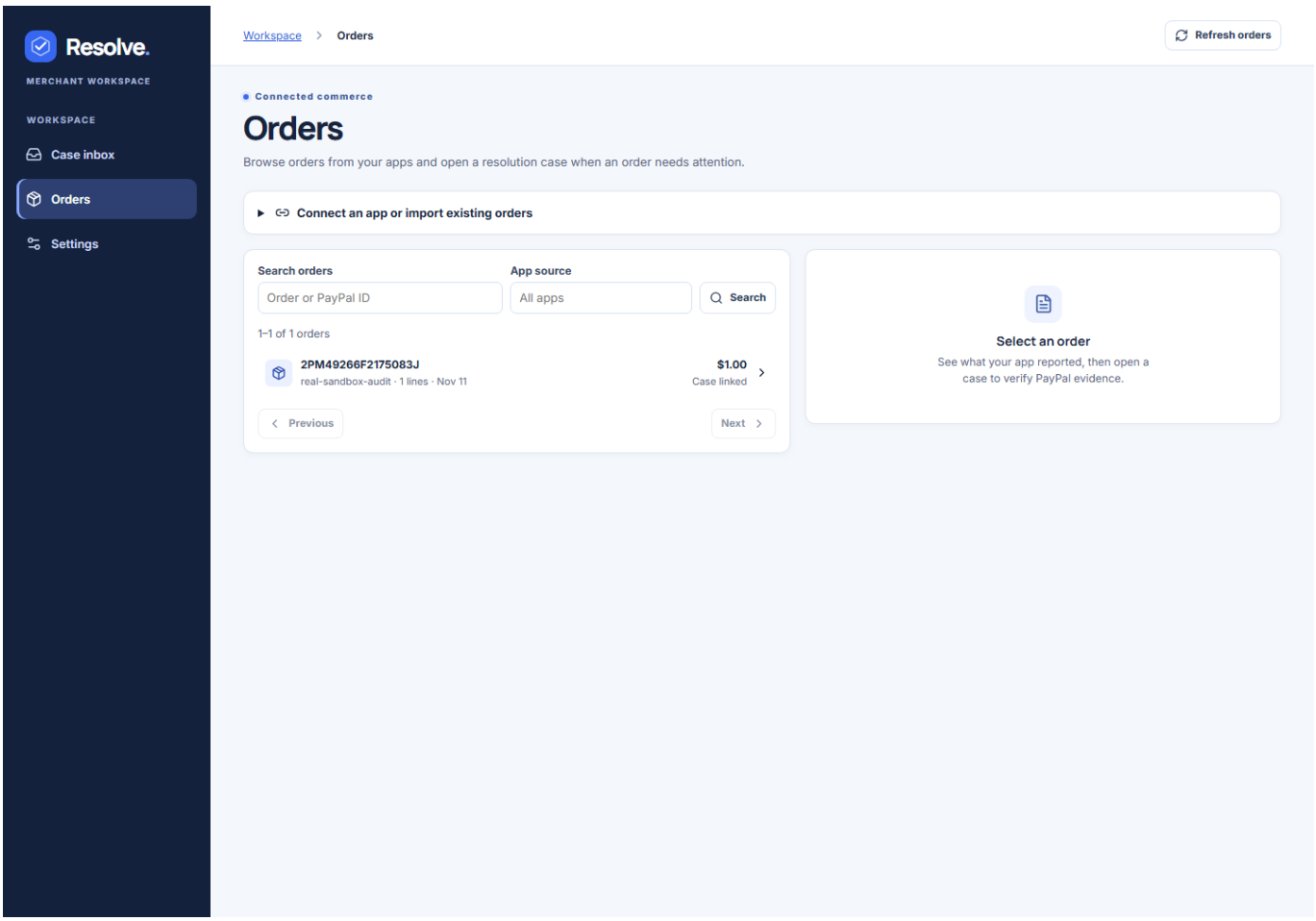
Use an isolated workspace and official Sandbox account for financial testing, with explicit approval before creating an order/capture/refund. Merchant-recorded observations are records, not a command to send money. A missing provider check displays “PayPal check needed”; confirmed, pending and outstanding totals remain unavailable until history is current.

Verified USD 1.00 Sandbox example

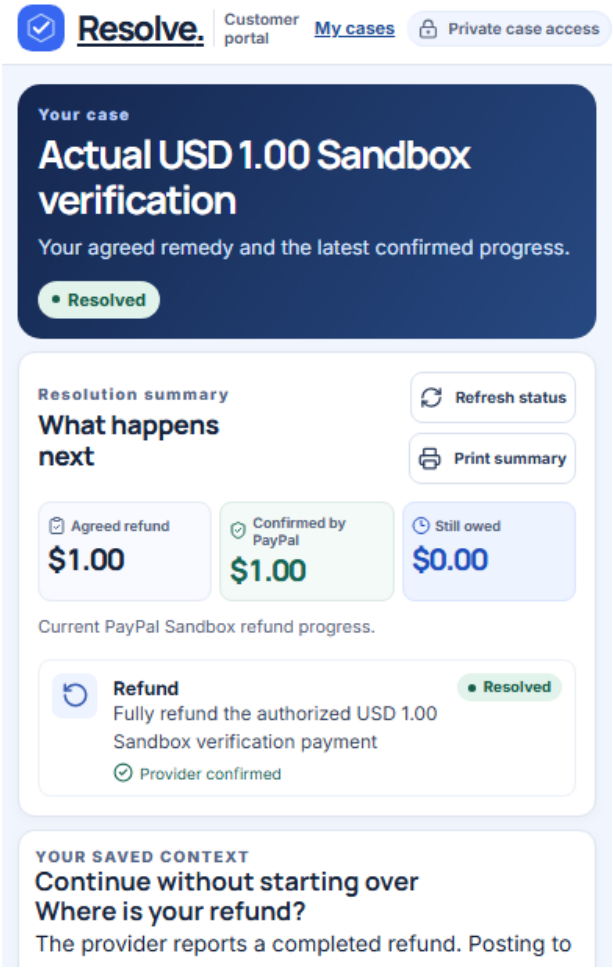
The owner authorized one Sandbox order/capture/full-refund workflow and privately approved the actual PayPal order. The existing checkout adapter created/captured it; the first-party SDK saved the order. The backend independently verified capture amount, currency, seller and refund history, then enforced a state-fingerprinted approval before separate execution. An independent PayPal refund read confirmed COMPLETED USD 1.00. The database holds one refund, reconciliation is fulfilled and remaining is USD 0.00. This is a real Sandbox result, not a production deployment or bank-credit claim.



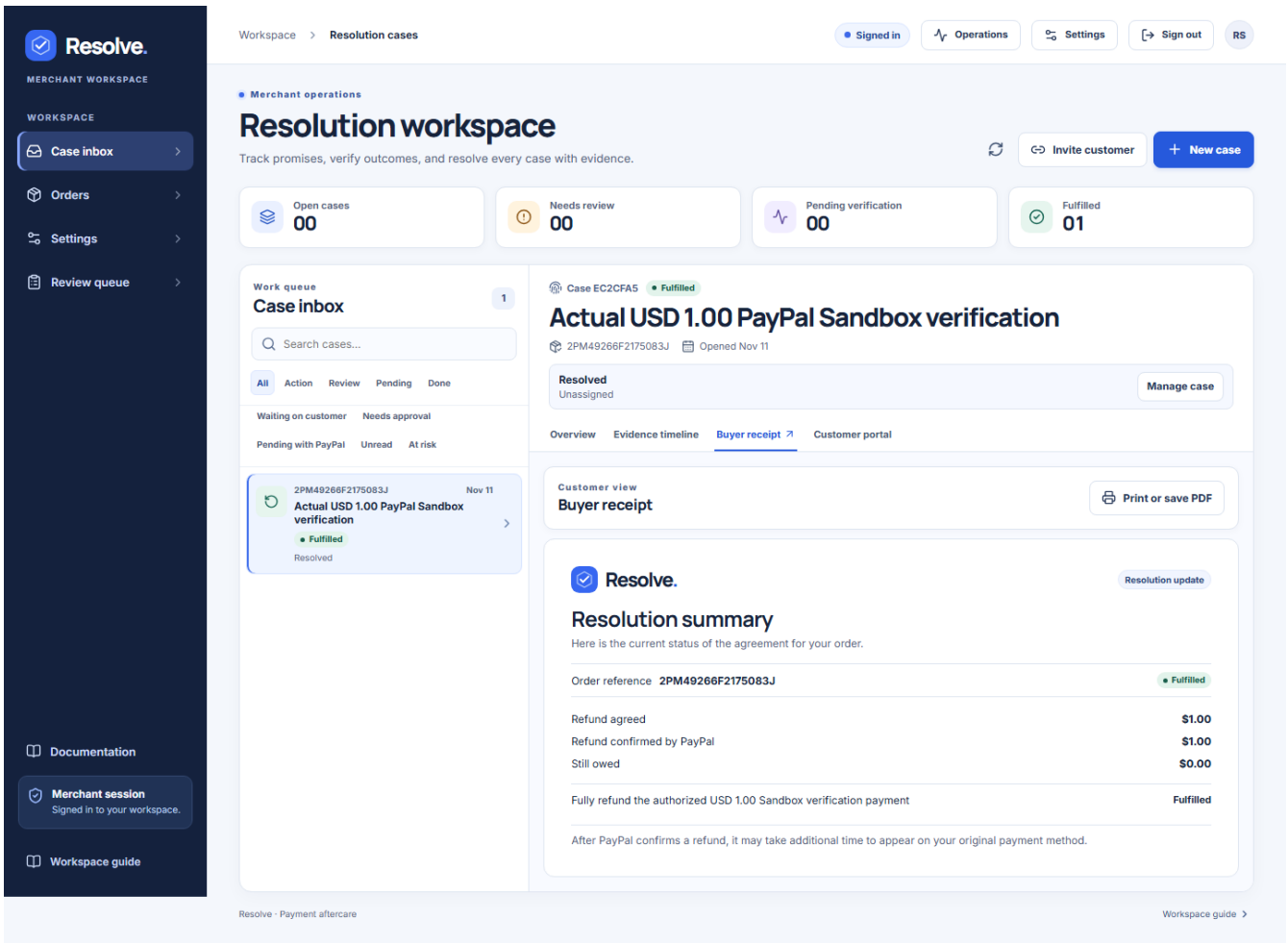
Actual official Sandbox case after independently verified full refund (upper portion; complete capture supplied separately)



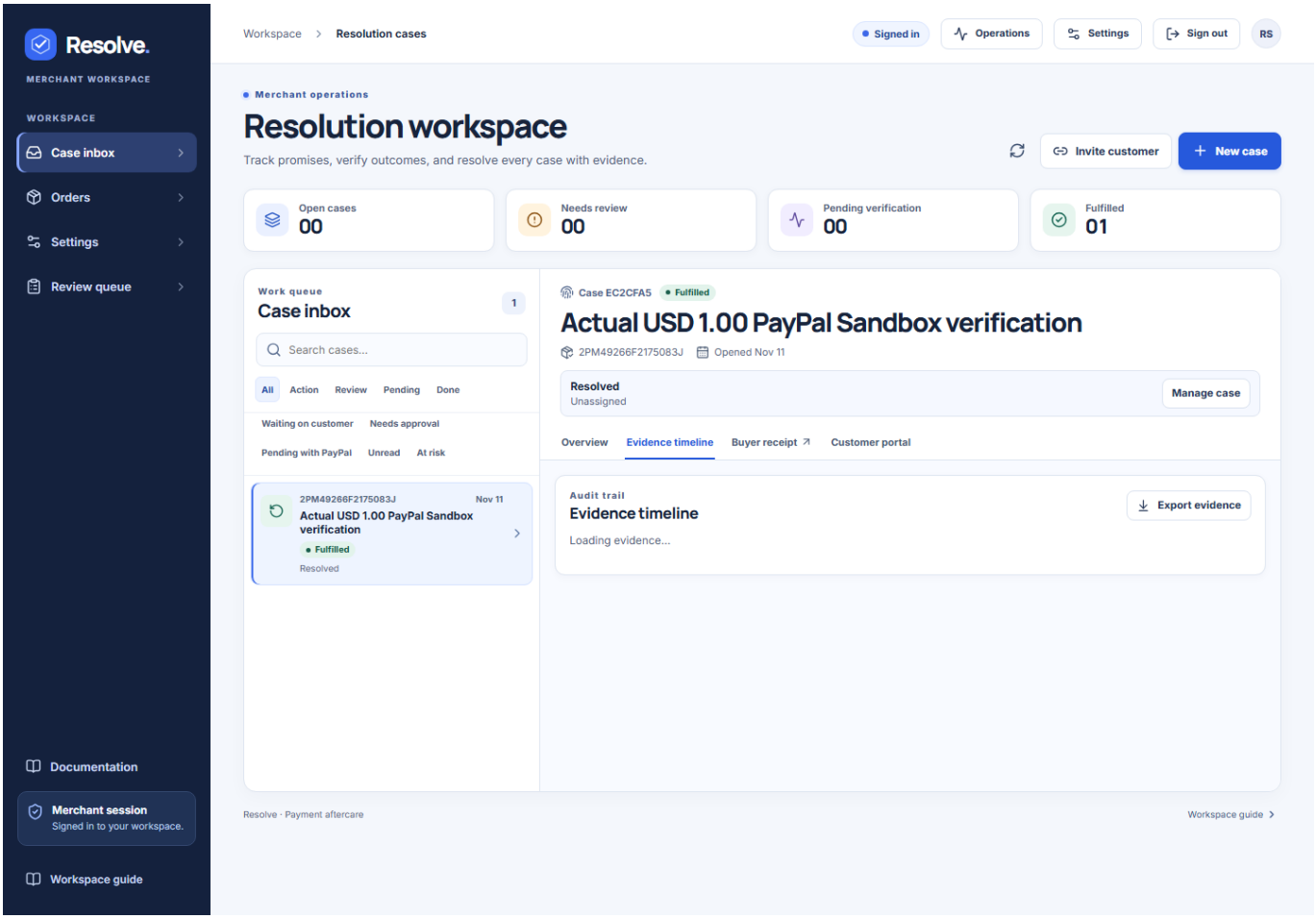
Actual order persisted by the SDK with its observed fully refunded state



Private buyer view showing the actual verified USD 1.00 refund on mobile (upper portion; complete capture supplied separately)



Receipt for the actual authorized Sandbox workflow



Timestamped evidence from the actual authorized Sandbox workflow

Complete journey: explicit merchant authorization → existing store adapter → official PayPal order → SDK → backend/database → private buyer PayPal approval → existing capture adapter → SDK update → backend capture/history verification → separate approval/execution → official refund → independent provider read → database/audit → merchant and buyer views. See `../testing/production-integration-report.md` for scope, statuses and missing credentials for other integrations.

Find and maintain records

Use inbox filters/search for cases, Orders search for registered orders and Operations for cross-case work. Select a result and verify its order reference before editing. Refresh after a successful mutation to confirm persistence. SDK keys and private links are revoked through their dedicated controls; they are not deleted by clearing browser history.

Manage account and integrations

Owners use Team for staff access and Connections for merchant provider configuration. Staff use their own accounts rather than shared full merchant keys. SDK keys belong on app servers; support keys require app-side order ownership checks. Optional AI proposals are advisory and must be reviewed before saving; investigation approval never authorizes a payment.

6. User roles and permissions

Role	Case work	Approve refund	Submit approved Sandbox refund	Review dispute package	Manage team/connections/keys
Owner / full legacy merchant key	Yes	Yes	Yes	Yes	Yes
Manager	Yes	Yes	No	Yes	No
Finance	Yes	Yes	Yes	No	No
Agent	Yes	No	No	No	No
Viewer	Read	No	No	No	No
Buyer	Own private portal only	No	No	No	No
Scoped SDK key	Granted server operations only	No	No	No	No

Finance/owners record refund traces; owners/managers/finance update dispute state; only owners change return policy. Some read-only sections may display controls that the backend denies for a restricted role. A visible control is not authorization. Staff emails currently belong to one workspace.

7. Troubleshooting

Problem	What to do
Cannot reach website	Check connection and supplied HTTPS URL; for local use start both services.
Backend unavailable / action fails	Wait, refresh once, then contact administrator; do not repeatedly submit a financial action.
Sign-in form stays disabled	Enable JavaScript and reload; check browser extensions.
Invalid password/key	Verify the chosen sign-in method. Ask owner/admin about access; no password reset UI exists.
Too many attempts	Wait the stated period (staff authentication throttle uses fifteen minutes) and verify credentials.
Workspace redirect after working	Session expired or was revoked. Sign in again.
Origin/address error	Use the exact configured host; administrator must align website origins.
No order/case search result	Check filters and registered reference; order synchronization may not have occurred.
Private link expired/revoked	Ask merchant for a replacement; do not share another buyer's link.
No My cases login option	Feature is disabled; use the merchant's private case link.
File upload rejected	Check supported file type and server-reported size limit; use the appropriate evidence or PDF-label control.
Refund blocked	Read current unmet conditions, acceptance, provider-history, pending balance and dispute hold. Obtain fresh evidence/approval rather than bypassing a gate.
Provider connection check fails	Owner should verify Sandbox account, scopes and saved connection; never expose the secret in a screenshot.
Imported comments missing from buyer portal	Private Zendesk sources intentionally stay in merchant evidence; compose a safe buyer message.
Return label unavailable	Merchant must attach an existing PDF label; Resolve does not purchase one.
Recorded completion but bank funds absent	Consult the merchant/provider using the trace; bank receipt is not independently verified.

8. Frequently asked questions

Is there a demo or fake fallback? No. Merchant and buyer workflows call the backend. Missing credentials or provider failures are shown honestly; they do not produce fabricated payment success.

Can Resolve submit live PayPal refunds? The implemented financial execution is Sandbox only, with explicit enablement and separate approval/submission.

Does accepting an offer automatically refund me? No. Acceptance records exact terms; the merchant still completes the authorized workflow.

Is a completed promise always verified by PayPal? No. Check the source label. Some milestones are merchant-recorded, and provider proof may be missing or stale.

Can I delete a case or reset my password here? Those general workflows are not implemented. Cases can close/reopen; ask the owner/admin about access recovery.

Can I use my customer session as a merchant? No. Customer and merchant permissions are separate.

Does a dispute package submit a PayPal dispute? No. It is a reviewed export for a merchant to handle through the appropriate external process.

Where is the production URL? No hosted deployment has been verified in this edition. Ask the project owner after deployment acceptance checks pass.

9. Support and contact information

No public support email, phone number or service-level agreement is configured in this repository. Buyers should contact the merchant through their case conversation or the merchant's existing support channel. Staff should contact the workspace owner; hosting issues go to the project administrator. Share the case ID, time, action and sanitized error text. Never send passwords, access keys, private links, provider secrets or raw customer evidence in public issue reports.

Screenshot provenance

All supplied screenshots are captured from the running application. Filenames include viewport width; the index records origin, scope, captions and hashes. Files 40-44 show independently verified official Sandbox evidence. Other screens use API-persisted QA data with providers disabled and unavailable verification clearly labelled. None came from provider mocks or generated artwork. Local captures do not establish production deployment. Before a hosted demonstration, rerun release checks and capture that deployed version.

SDK INTEGRATION AND IMPLEMENTATION GUIDE

This guide describes the checked-out implementation, not generic vendor examples. Versions come from lock/package files. Installation initializes code; only observed requests to the real service establish integration verification. Reasons below describe the implementation's fit unless an earlier engineering decision explicitly records original selection.

SDKs versus HTTP integrations

Resolve's Node SDK is first-party. Groq is an official vendor SDK. PayPal, Shopify, EasyPost, Zendesk, Resend and OpenRouter use direct official HTTP APIs through HTTPX/custom adapters; there is no installed PayPal/Shopify/EasyPost/Resend/OpenRouter vendor SDK to claim. PyJWT, Fernet and PostgreSQL clients support

security/storage. React/Next.js/FastAPI/Pydantic are application frameworks or validation libraries, not evidence that an external service has been contacted.

Environment and secret boundary

Ordinary backend variables are loaded by app/settings.py from process environment or backend/.env. AI settings deliberately read process environment only; a private .env.ai.ps1 file is not activated until explicitly sourced. Frontend server variables belong in frontend/.env.local or the host's secret store. Store/SDK variables belong in the integrating server. None belong in NEXT_PUBLIC_* or the browser. Never copy real credential values into these snippets, Git, reports or recordings. See the integration report for missing names and authorization/resource blockers.

Resolve Evidence SDK

Item	This project's implementation
1. SDK name	@resolve-paypal/sdk
2. Provider	This repository (first-party; not a PayPal SDK)
3. Version	0.7.0
4. Selection / implementation fit	Typed server-only access, generated contract, retry identity and restricted app-source support grants fit the existing app integration; there is no runtime dependency.
5. Installation	npm --prefix sdk/typescript ci; npm --prefix sdk/typescript run build; npm --prefix sdk/typescript pack; install the local tarball in the integrating Node server. The package is not claimed to be published on npm.
6. Initialization file	sdk/typescript/src/index.ts; examples/checkout-store/src/server.mjs
7. Authentication	Opaque Resolve bearer key, independently of PayPal OAuth. Browser cookies alone are not SDK credentials.
8. Required credentials	RESOLVE_API_URL, RESOLVE_API_KEY; source-bound support grant for support actions
9. Configuration	RESOLVE_APP_SOURCE; timeoutMs; retryAttempts; HTTPS or trusted loopback URL
10. Methods used	createResolve, syncOrder, openIssue, getOrderResolution, createSupportLink, lookupIssue, pullEvents, getRefundJourney, getReturnEligibility, getSupportContext, requestOrderReturn, downloadReturnLabel; full exported method index below
11. Request / response handling	Node fetch, bounded timeouts, typed generated JSON, fixed relative paths and no redirects
12. Error handling	ResolveApiError/ResolveTransportError; no raw error body is surfaced. Retries retain write identity only for supported safe operations; link rotation is not blindly retried.
13. Backend integration	Store server creates one ResolveClient. /api/orders actions call it after checking receipt-session ownership.
14. Frontend integration	No merchant key goes into browser code. public/app.js calls only its own store routes; portal links are explicit user actions.
15. Database interaction	FastAPI owns Resolve persistence; store_orders holds the app receipt and Resolve case reference. SDK does not secretly read a merchant DB.
16. Complete execution flow	Buyer action -> store route/session check -> Resolve SDK -> FastAPI scoped grant -> transaction/evidence adapter -> JSON -> store route -> buyer UI

HTTPX and real provider adapters

Item	This project's implementation
1. SDK name	httpx (generic HTTP transport, not a provider SDK)
2. Provider	Encode / HTTPX community; providers supply official HTTP APIs
3. Version	0.28.1
4. Selection / implementation fit	Existing adapters validate explicit provider responses, fixed hosts, seller identity and bounds; no additional vendor SDK is installed for PayPal, Shopify, EasyPost, Zendesk, Resend or OpenRouter.
5. Installation	cd backend; uv sync --locked
6. Initialization file	app/paypal.py; app/resolution_providers.py; app/helpdesk.py; app/platform_imports.py; app/customer_email.py; app/inference.py
7. Authentication	PayPal client_credentials + Bearer; Shopify X-Shopify-Access-Token; EasyPost HTTP Basic API key; Zendesk/Resend/OpenRouter Bearer
8. Required credentials	Provider-specific variables in the adapter matrix below; optional encrypted per-workspace connection
9. Configuration	Ordinary settings: process environment/backend .env. AI: process environment only. Provider reads/writes have separate enablement.
10. Methods used	httpx.Client.request, get, post, stream; PayPalSandboxClient.read_capture/read_refund/read_refund_history; ProofProviders.read; provider_json; deliver
11. Request / response handling	No redirects; normal provider clients set trust_env=False; responses are bounded and structurally validated, with monetary strings checked using Decimal/Pydantic.
12. Error handling	PayPal timeout 504, transport/invalid response 502, rejected app credentials/unavailable/rate limit 503; business conflicts 409. Provider body/credentials are not displayed.
13. Backend integration	FastAPI routes call ownership-scoped services; separate workers recover jobs and refresh authorized evidence.
14. Frontend integration	Next.js forwards same-origin requests server-side. Client forms receive sanitized detail/status; no public provider secrets.
15. Database interaction	Only validated observations become source-labelled records in transactions. Manual notes and provider observations remain separate.
16. Complete execution flow	Merchant click -> /api/resolve gateway -> backend authorization -> service -> real HTTPS provider -> validation -> database/audit -> response -> refreshed UI

Groq Python SDK

Item	This project's implementation
1. SDK name	groq
2. Provider	Groq (official Python SDK)
3. Version	1.7.0
4. Selection / implementation fit	The implemented extraction path uses the official client and its typed API exceptions. Existing inference jobs also have an HTTPX adapter; that is a separate code path, not evidence that the SDK ran.
5. Installation	cd backend; uv sync --locked
6. Initialization file	backend/app/ai_routes.py:get_ai_client; backend/app/platform_imports.py

Item	This project's implementation
7. Authentication	API key passed on the server to groq.Groq
8. Required credentials	RESOLVE_AI_GROQ_KEY
9. Configuration	RESOLVE_EXTRACTION_ENABLED; RESOLVE_AI_DATA_POLICY; RESOLVE_EXTRACTION_MODEL; timeout/daily token limits. Environment only.
10. Methods used	Groq(...), client.chat.completions.create(...), close; see ai_extraction.py
11. Request / response handling	A minimized transcript and strict structured schema are submitted; verbatim merchant quote, source message and amount gates validate proposals.
12. Error handling	APITimeoutError, APIConnectionError and APIStatusError become needs_review. Budget/privacy rejection blocks calls. A model proposal is not saved or paid automatically.
13. Backend integration	POST case ai/extract-promises and explain-next-action use dependency-injected SDK clients; importing Zendesk can use the same optional path.
14. Frontend integration	Promise suggestions appear for merchant review. Choosing a proposal fills the normal form; saving is a separate authenticated action.
15. Database interaction	Usage counters and reviewed promises are stored; extraction itself is advisory. It never authorizes a refund.
16. Complete execution flow	Merchant requests analysis -> privacy/budget gate -> Groq SDK -> real Groq endpoint -> structured/quote validation -> proposal -> merchant-reviewed save. Paid/production-key calls remain blocked without explicit authorization.

PostgreSQL Python driver and pool

Item	This project's implementation
1. SDK name	psycopg[binary] and psycopg-pool
2. Provider	Psycopg project (database driver, not a Neon SDK)
3. Version	3.3.8 / 3.3.3
4. Selection / implementation fit	The existing SQLite repository abstraction is retained; PostgreSQL adds durable hosted storage and bounded reusable connections without a second ORM.
5. Installation	cd backend; uv sync --locked
6. Initialization file	backend/app/postgres.py:PostgresStore; backend/app/storage.py:configured_store
7. Authentication	Database connection URL / PostgreSQL role credentials
8. Required credentials	RESOLVE_DATABASE_URL (private); provider-supplied TLS options
9. Configuration	Per-process connection pool; schema and transaction isolation managed by PostgresStore
10. Methods used	ConnectionPool.connection, execute, fetchone/fetchall, transactions, sql.Identifier
11. Request / response handling	Parameterized SQL values and quoted identifiers; read-only repeatable-read snapshots and serialized writes preserve authoritative reconciliation state.
12. Error handling	Operational/pool/timeouts become sanitized 503; expected integrity/business conflicts retain domain errors.
13. Backend integration	API, CLI and all workers use the same configured store. Lifespan closes pools.

Item	This project's implementation
14. Frontend integration	No database connection string is shipped to browser JavaScript.
15. Database interaction	Checksum-verified immutable migrations, case/order/event/evidence/session transactions, shared balances and idempotency constraints.
16. Complete execution flow	API authorization -> store transaction -> PostgreSQL driver/pool -> SQL constraints/snapshot -> mapped domain response. Disposable DB validation does not verify the configured hosted account.

Node PostgreSQL adapter / consumer

Item	This project's implementation
1. SDK name	pg-compatible pool supplied by the integrating app
2. Provider	node-postgres community; Resolve connector is first-party
3. Version	pg 8.23.1
4. Selection / implementation fit	Injected read/query interfaces let the merchant own DB authentication and approved views; the SDK does not request arbitrary customer tables.
5. Installation	Install pg in the integrating server when using these optional adapters; build the local SDK as above.
6. Initialization file	sdk/typescript/src/commerce.ts; sdk/typescript/src/events.ts; SDK PostgreSQL examples
7. Authentication	App-owned pool/role connectionString; source-bound Resolve key separately
8. Required credentials	App database URL in the integrating server secret store; Resolve server variables separately
9. Configuration	Approved projection views/queries and stream consumer checkpoint table; caller owns pool configuration
10. Methods used	PostgresCommerce.sync/backfill; PostgresResolutionConsumer.consume; pool.connect/query/release; transactional callback
11. Request / response handling	Selected fields are projected into versioned order snapshots. Stream consumers apply callbacks/checkpoints atomically.
12. Error handling	Invalid source shape/query raises CommerceSourceError. Transactions roll back failed consumers; duplicate/older events do not reapply effects.
13. Backend integration	Merchant app server reads its authorized views then sends snapshots through ResolveClient. Consumer runs in the app process.
14. Frontend integration	No browser DB credentials or direct browser SQL.
15. Database interaction	App views are read-only; the optional app event consumer owns its own checkpoint transaction/table, separate from Resolve migrations.
16. Complete execution flow	App DB view -> PostgresCommerce -> SDK -> Resolve ingestion -> evidence comparison; event feed -> app transaction -> callback + checkpoint commit/rollback

Nodemailer SMTP transport

Item	This project's implementation
1. SDK name	nodemailer
2. Provider	Nodemailer project (SMTP client, not Gmail/Resend SDK)
3. Version	10.3.1
4. Selection / implementation fit	The existing follow-up worker can use standard SMTP through a small Node subprocess while keeping secrets out of process arguments and output.
5. Installation	npm ci --prefix backend/mailer; install Node on the follow-up worker; retain Python backend dependencies
6. Initialization file	backend/mailer/send.mjs; backend/app/customer_smtp.py
7. Authentication	SMTP auth user/password over implicit TLS or required STARTTLS
8. Required credentials	RESOLVE_SMTP_HOST/PORT/SECURE/USER/PASSWORD/FROM; customer signing key/origin/merchant binding
9. Configuration	RESOLVE_CUSTOMER_EMAIL_ENABLED=true; RESOLVE_CUSTOMER_EMAIL_TRANSPORT=smtp, only after authorized delivery testing
10. Methods used	nodemailer.createTransport; transport.sendMail; transport.close
11. Request / response handling	Python sends a JSON payload on stdin; Node returns only accepted/retry/failed. TLS minimum 1.2; file/URL access disabled; bounded connection/greeting/socket timeouts.
12. Error handling	Explicit transient SMTP rejections may retry; ambiguous transfer failure is not auto-retried as if nothing was sent. stderr is suppressed to avoid credential leakage.
13. Backend integration	followup_worker -> customer_email.deliver -> send_smtp -> Node transport -> SMTP
14. Frontend integration	UI queues or requests mail; a queued notice is not a delivery claim.
15. Database interaction	Durable outbox keeps intended notice identity/status. SMTP acceptance is not inbox receipt.
16. Complete execution flow	Authenticated action -> outbox transaction -> separate worker -> Nodemailer -> SMTP acceptance/failure -> outbox state -> UI status. No real message is sent without authorization.

PyJWT / Google token validation

Item	This project's implementation
1. SDK name	PyJWT with crypto
2. Provider	PyJWT community; Google supplies OIDC/JWKS (no Google client SDK installed)
3. Version	2.15.1
4. Selection / implementation fit	The implementation uses OIDC authorization code/PKCE and verifies signatures/claims locally; adding a second browser identity SDK was unnecessary for this architecture.
5. Installation	cd backend; uv sync --locked
6. Initialization file	backend/app/customer_google.py:_verify; frontend/app/api/customer-account/[...path]/route.ts
7. Authentication	OAuth client ID/secret, one-use state, PKCE verifier; signed RS256 ID token
8. Required credentials	RESOLVE_GOOGLE_CLIENT_ID / SECRET and exact customer HTTPS origin; Google-enabled flag

Item	This project's implementation
9. Configuration	Registered /api/customer-account/google/callback; server-only secret, exact issuer/audience and authoritative email checks
10. Methods used	jwt.get_unverified_header (key selection only), PyJWK.from_dict, jwt.decode
11. Request / response handling	JWKS key and token issuer/audience/expiry are verified; unverified header claims are never trusted as identity.
12. Error handling	Bad state/code/signature/claims/address authority produce sanitized failure; missing config is disabled/unavailable, not a fabricated login.
13. Backend integration	CustomerAccounts binds verified email to invited cases and isolated customer sessions.
14. Frontend integration	Buyer chooses Google, approves on Google, returns to server callback; no Google secret appears in the browser.
15. Database interaction	One-time flow records, customer sessions and mailbox-bound links are durable and expiring.
16. Complete execution flow	Buyer -> state/PKCE -> official Google consent/token/JWKS -> PyJWT verify -> invited mailbox lookup -> customer cookie -> own cases

Fernet connection encryption

Item	This project's implementation
1. SDK name	cryptography.fernet.Fernet
2. Provider	Python Cryptographic Authority
3. Version	50.0.2
4. Selection / implementation fit	A stable symmetric key protects per-workspace provider secrets; it fits the existing server-side connection registry and reversible credential use.
5. Installation	cd backend; uv sync --locked
6. Initialization file	backend/app/platform_connections.py:Connections.cipher/decrypt/save
7. Authentication	Not provider authentication; local authenticated encryption protects stored credentials
8. Required credentials	RESOLVE_PLATFORM_ENCRYPTION_KEY (shared stable Fernet key)
9. Configuration	Same key on API/workers and private backups; do not regenerate for existing encrypted rows
10. Methods used	Fernet(...), encrypt, decrypt; InvalidToken handling
11. Request / response handling	Only redacted connection configuration/status returns to clients. Decrypted secrets stay in server memory for a scoped provider request.
12. Error handling	Missing/invalid key or InvalidToken returns sanitized 503. Wrong keys do not silently reset credentials.
13. Backend integration	Connections enforces owner role, expected revision and merchant/provider ownership.
14. Frontend integration	Owners submit secret fields over the gateway; listing never returns plaintext secrets.
15. Database interaction	platform_connections stores credentials_ciphertext, safe config and revision/check status; connection changes invalidate stale financial assumptions.
16. Complete execution flow	Owner form -> backend owner/revision gate -> Fernet encrypt -> DB; provider check -> decrypt -> real API -> safe status

Actual source excerpts

These are extracted from the current source files. Class methods are shown as methods, not standalone executable programs. Read the named files for surrounding imports/types. No configured values or customer data are included.

Resolve client initialization

Source: `sdk/typescript/src/index.ts`

```
  constructor(options: ResolveClientOptions) {
    if (typeof window !== "undefined" && typeof window.document !== "undefined") {
      throw new TypeError("Use ResolveClient on the server; never put a merchant key in browser code.");
    }
    if (typeof options.apiKey !== "string" || !/^rslv_[A-Za-z0-9_-]{43}$/.test(options.apiKey)) {
      throw new TypeError("Set a valid Resolve merchant access key.");
    }
    this.#base = apiBase(options.baseUrl);
    this.#apiKey = options.apiKey;
    this.#source = options.source ?? "my-app";
    if (!/^[A-Za-z0-9][A-Za-z0-9_.-]{0,63}$/.test(this.#source)) {
      throw new TypeError("Set a valid RESOLVE_APP_SOURCE, such as my-store.");
    }
    this.#retryAttempts = options.retryAttempts ?? 3;
    if (!Number.isInteger(this.#retryAttempts) || this.#retryAttempts < 1 || this.#retryAttempts > 4)
throw new TypeError("Use retryAttempts between 1 and 4.");
    this.#timeoutMs = options.timeoutMs ?? 10_000;
    if (!Number.isInteger(this.#timeoutMs) || this.#timeoutMs < 1 || this.#timeoutMs > 120_000) {
      throw new TypeError("Set timeoutMs to an integer between 1 and 120,000.");
    }
  }
```

Order synchronization

Source: `sdk/typescript/src/index.ts`

```
  syncOrder(order: SimpleOrder, options?: RequestOptions): Promise<CommerceOrderReceipt> {
    const { id, version, status, ...fields } = order;
    return this.registerCommerceOrder({
      ...fields, source: this.#source, order_reference: id, revision: version ?? 1,
      ...(status !== undefined ? { order_status: status } : {}),
    }, options);
  }
```

Order support call

Source: `sdk/typescript/src/index.ts`

```
  openIssue(input: OpenIssueInput, options?: RequestOptions): Promise<IssueReceipt> {
    const { orderId, requestId, reason, message, requestedResolution } = input;
    return this.#request("POST", "/aftercare/issues", options, {
      source: this.#source, order_reference: orderId, request_id: requestId, reason, message,
      ...(requestedResolution !== undefined ? { requested_resolution: requestedResolution } : {}),
    });
  }
```

PayPal transport initialization

Source: `backend/app/paypal.py`

```
    def __init__(self, settings: Settings, http_client: httpx.Client | None = None):
        self.settings = settings
        # Test injection configures an actual local HTTP proxy, never a runtime base URL.
        self.http = (
            http_client
```

```
            if http_client is not None
            else httpx.Client(
                timeout=httpx.Timeout(10.0, connect=5.0),
                follow_redirects=False,
                trust_env=False,
            )
        )
```

PayPal authentication

Source: `backend/app/paypal.py`

```
    def _access_token(self, settings: Settings) -> str:
        data = self._json_request(
            "POST",
            "/v1/oauth2/token",
            token_request=True,
            auth=httpx.BasicAuth(
                settings.paypal_sandbox_client_id.get_secret_value(),
                settings.paypal_sandbox_client_secret.get_secret_value(),
            ),
            data={"grant_type": "client_credentials"},
            headers={"Accept": "application/json"},
        )
        try:
            token = _OAuthToken.model_validate(data)
        except ValidationError:
            raise StoreError(
                502, "PayPal Sandbox returned an invalid token response"
            ) from None
        return token.access_token.get_secret_value()
```

Provider response/error mapping

Source: `backend/app/paypal.py`

```
    def _json_request(
        self,
        method,
        path,
        *,
        token_request=False,
        not_found_detail="PayPal Sandbox capture was not found",
        **kwargs,
    ):
        try:
            response = self.http.request(
                method, SANDBOX_BASE_URL + path, follow_redirects=False, **kwargs
            )
        except httpx.TimeoutException:
            raise StoreError(504, "PayPal Sandbox request timed out") from None
        except httpx.HTTPError:
            raise StoreError(502, "PayPal Sandbox connection failed") from None
        if response.status_code == 404 and not token_request:
            raise StoreError(404, not_found_detail)
        if response.status_code in (401, 403):
            if token_request:
                raise StoreError(503, "PayPal Sandbox credentials were rejected")
            raise StoreError(502, "PayPal Sandbox could not authorize the capture read")
        if response.status_code == 429 or response.status_code >= 500:
            raise StoreError(503, "PayPal Sandbox is temporarily unavailable")
        if response.status_code != 200:
            raise StoreError(502, "PayPal Sandbox returned an unexpected response")
        try:
            return response.json()
        except ValueError:
            raise StoreError(
                502, "PayPal Sandbox returned an invalid response"
            ) from None
```

Groq SDK initialization

Source: backend/app/ai_routes.py

```
def get_ai_client(settings: Annotated[ExtractionSettings, Depends(get_ai_settings)]):
    if not settings.enabled or settings.groq_key is None:
        yield None
        return
    client = groq.Groq(
        api_key=settings.groq_key.get_secret_value(),
        timeout=settings.timeout_seconds,
        max_retries=1,
    )
    try:
        yield client
    finally:
        client.close()
```

Fernet initialization

Source: backend/app/platform_connections.py

```
    def cipher(self):
        try:
            return Fernet(
                self.settings.platform_encryption_key.get_secret_value().encode()
            )
        except (AttributeError, ValueError):
            raise StoreError(
                503,
                "Set RESOLVE_PLATFORM_ENCRYPTION_KEY to a stable Fernet key before connecting accounts",
            ) from None
```

Google verification

Source: backend/app/customer_google.py

```
    def _verify(config, http, code, verifier):
        try:
            client_id = config.client_id.get_secret_value()
            result = http.post(
                TOKEN,
                data={
                    "client_id": client_id,
                    "client_secret": config.client_secret.get_secret_value(),
                    "redirect_uri": config.callback,
                    "grant_type": "authorization_code",
                    "code": code,
                    "code_verifier": verifier,
                },
            )
            if result.status_code == 400:
                raise StoreError(400, "Google did not accept this sign-in attempt")
            result.raise_for_status()
            grant = result.json()
            token = grant.get("id_token") if isinstance(grant, dict) else None
            if not isinstance(token, str) or not 0 < len(token) <= 8192:
                raise ValueError
            header = jwt.get_unverified_header(token)
            kid = header.get("kid")
            if (
                header.get("alg") != "RS256"
                or not isinstance(kid, str)
                or not 0 < len(kid) <= 128
            ):
                raise ValueError
            response = http.get(KEYS)
            response.raise_for_status()
            jwks = response.json()
            keys = jwks.get("keys") if isinstance(jwks, dict) else None
```

```
            if not isinstance(keys, list):
                raise ValueError
            matches = [
                key
                for key in keys
                if isinstance(key, dict)
                and key.get("kid") == kid
                and key.get("kty") == "RSA"
            ]
            if len(matches) != 1:
                raise ValueError
            claims = jwt.decode(
                token,
                jwt.PyJWK.from_dict(matches[0], algorithm="RS256").key,
                algorithms=["RS256"],
                audience=client_id,
                issuer=("https://accounts.google.com", "accounts.google.com"),
                options={
                    "strict_aud": True,
                    "require": [
                        "sub",
                        "iss",
                        "aud",
                        "exp",
                        "iat",
                        "nonce",
                        "email",
                        "email_verified",
                    ],
                },
            )
            if not claims["sub"] or claims.get("azp", client_id) != client_id:
                raise ValueError
            return claims
        except (httpx.HTTPError, jwt.PyJWTError, TypeError, ValueError):
            raise StoreError(503, "Google sign-in could not be verified") from None
```

SMTP transport

Source: `backend/mailer/send.mjs`

```
import { readFileSync } from "node:fs";
import nodemailer from "nodemailer";

let status = "failed";
let transport;
try {
  const { host, port, secure, user, password, message } = JSON.parse(readFileSync(0, "utf8"));
  transport = nodemailer.createTransport({
    host,
    port,
    secure,
    requireTLS: !secure,
    auth: { user, pass: password },
    tls: { minVersion: "TLSv1.2" },
    connectionTimeout: 8_000,
    greetingTimeout: 8_000,
    socketTimeout: 10_000,
    disableFileAccess: true,
    disableUrlAccess: true,
  });
  const result = await transport.sendMail(message);
  if (result.accepted.length === 1 && !result.rejected.length) status = "accepted";
} catch (error) {
  // An explicit transient SMTP rejection is safe to retry; an interrupted
  // transfer may already have been accepted and is never retried automatically.
  if (Number.isInteger(error?.responseCode) && error.responseCode >= 400 &&
      error.responseCode < 500) status = "retry";
} finally {
  transport?.close();
}
process.stdout.write(JSON.stringify({ status }));
```

Provider adapter configuration and real request flow

Integration	Actual implementation / protocol	Configuration and effect boundary
PayPal Sandbox payments	app/paypal.py, executions.py, paypal_history.py; OAuth, capture/order/refund GETs, separately approved capture-refund POST, signed webhook verification	RESOLVE_PAYPAL_SANDBOX_CLIENT_ID/SECRET/MERCHANT_ID; RESOLVE_PAYPAL_RESOLVE_MERCHANT_ID; read/write/partial flags and webhook ID. Fixed Sandbox host; user confirmation required for the audit order/capture/refund.
PayPal merchant identity	app/merchant_auth.py:PayPalIdentity; authorization code and subject read	RESOLVE_PAYPAL_LOGIN_CLIENT_ID/SECRET/ENVIRONMENT and exact merchant origin; linked-account consent is separate from payment authorization.
Shopify	resolution_providers.py:_shopify_read; Admin GraphQL with fixed shop hostname	RESOLVE_SHOPIFY_SHOP_DOMAIN / ADMIN_TOKEN / RESOLUTION_CONNECTOR_MERCHANT_ID, or encrypted owning connection; read evidence only. No order mutation or fulfillment creation is implemented.
EasyPost	resolution_providers.py:tracking; GET existing tracker with Basic auth	RESOLVE_EASYPOST_API_KEY and connector binding or owning connection. No shipment/label purchase/tracking-ID creation is implemented.
Zendesk	helpdesk.py, platform_imports.py and operations_worker; ticket/comment reads and signed webhook jobs	RESOLVE_ZENDESK_SUBDOMAIN / OAUTH_TOKEN / HELPDESK_MERCHANT_ID or encrypted connection/webhook_secret. Private comments remain private.
Resend	customer_email.py:deliver; POST /emails with stable idempotency identity	RESOLVE_RESEND_API_KEY / FROM plus customer signing key/origin/merchant binding. Queued is not sent; API acceptance is not inbox delivery.
OpenRouter	inference.py:OpenRouterProvider; structured chat via HTTPX	RESOLVE_AI_OPENROUTER_KEY / ENABLED / DATA_POLICY and routing settings. No production-key/model call is authorized by the read-only audit.
SMTP	customer_smtp.py -> send.mjs -> real SMTP	Credentials/configuration in SDK table; a verified sender/recipient and explicit delivery authorization are still required.

Full first-party SDK method index

• `syncOrder`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `openIssue`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getOrderResolution`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `createSupportLink`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getRefundJourney`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getSupportContext`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `lookupIssue`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `acknowledgeRefundReceived`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `pullEvents`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getReturnEligibility`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getOrderReturns`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `requestOrderReturn`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getReturnGuide`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `downloadReturnLabel`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `saveReturnPolicy`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `recordDispute`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `recordRefundTrace`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `recordDisputeEvidence`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `saveReturnGuide`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `uploadReturnLabel`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `recordReturnInspection`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `registerCommerceOrder`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `syncCommerceOrder`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getCommerceOrder`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getCommerceContact`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `refreshCommerceOrder`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• connectPostgres: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• getMerchant: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• createCase: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• upsertCase: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• linkRecord: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• addEvidence: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• refreshEvidence: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• getInvestigations: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• getOperationsDashboard: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• getMissedPromises: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• scanMissedPromises: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• importZendeskTicket: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• confirmSupportImport: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• publishResolutionOffer: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• getReturnWorkflow: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• requestReturn: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• updateReturn: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• prepareDisputePackage: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• reviewDisputePackage: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• reviewInvestigation: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• getCase: implementation in sdk/typescript/src/index.ts; generated request/response types come from postman/contracts/resolve.openapi.json.

• `listCases`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getCaseTimeline`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getTimeline`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getEvidenceBundle`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `issueCustomerLink`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `createBuyerLink`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `getCustomerLink`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

• `revokeCustomerLink`: implementation in `sdk/typescript/src/index.ts`; generated request/response types come from `postman/contracts/resolve.openapi.json`.

Practical implemented flows

Store order, payment and aftercare

1. public/app.js submits cart items and a request identity to the store server. The server quotes catalog prices and binds the receipt to the HttpOnly session.

2. SandboxPayPal.create sends an actual PayPal order request and validates the approval link. Buyer consent happens on PayPal; it is not simulated by the application.

3. The callback receipt uses SandboxPayPal.capture, which first reads the order and reuses capture identity. It validates amount, currency, seller, reference and capture count before saving store_orders.

4. server.mjs sync calls ResolveClient to register/sync the actual order snapshot. Resolve stores the revision and reads configured provider evidence only through authorized services.

5. The buyer submits a support request. The store verifies receipt ownership, then calls openIssue. Duplicate request IDs retain the same issue receipt. getOrderResolution/getSupportContext/getRefundJourney read safe status; createSupportLink grants the explicit private portal.

6. Failed/missing configuration stays a failure or unavailable state. There is no product DemoStore or fabricated fallback order/refund state. Controlled doubles are allowed only in isolated tests and never establish real-service VERIFIED status.

Merchant evidence to refund

Merchant workspace -> same-origin Next gateway -> merchant/role gate -> current promise/acceptance/condition checks -> fresh real Sandbox capture/history -> exact state-bound approval -> separate execution -> provider identity/amount check -> durable request/outcome -> independent refund read -> reconciliation and safe buyer update. A provider timeout can mean an unknown outcome; recovery reads the existing request instead of treating it as a new refund.

Current verification boundary

Consult `docs/testing/production-integration-report.md` for actual results, endpoint/status evidence and missing credentials. Real OAuth/order creation are distinct from completed buyer approval/capture/refund. Local database/SDK success does not verify hosted PostgreSQL, Render deployment, a real mailbox or a carrier account.

Official references used to check protocol terminology

- PayPal authentication
- Groq Python quickstart
- Google OpenID Connect
- Nodemailer SMTP